Rockchip eDP 接口开发指南

文件标识：RK-YH-YF-804

发布版本：V2.3.0

日期：2025-11-20

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文描述了 Rockchip 平台 eDP 接口的使用说明和常用问题的解决办法，本文档将不定期进行更新或修改。

产品版本

芯片名称	内核版本
RK3288	Linux kernel 4.4 及以上内核
RK3399	Linux kernel 4.4 及以上内核
RK356x	Linux kernel 4.19 及以上内核
RK3588	Linux kernel 5.10 及以上内核
RK3576	Linux kernel 6.1 及以上内核

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	吴小平	2024-04-16	初始版本
V1.1.0	吴小平	2025-04-10	添加 FAQ
V2.0.0	丁凌崧	2025-05-10	内容补充和格式优化
V2.1.0	丁凌崧/张嘉津	2025-05-25	《常见问题及分析方法》章节合入 FAQ
V2.2.0	张嘉津	2025-07-28	《Panel DTS 配置》章节内容补充和优化
V2.3.0	丁凌崧	2025-11-20	《PSR 模式》章节内容更新

目录

Rockchip eDP 接口开发指南

1. 概述
2. eDP 基础介绍
   - 2.1 DRM 显示框架
   - 2.2 eDP 接口系统框架
   - 2.3 eDP Link Training
3. Rockchip 平台 eDP 简介
   - 3.1 功能特性
   - 3.2 eDP 与 VOP 连接关系
   - 3.3 驱动文件
4. 软件配置
   - 4.1 模块使能
     - 4.1.1 Controller DTS 配置
       - 4.1.1.1 HPD 配置
       - 4.1.1.2 Panel 场景
       - 4.1.1.3 Monitor 场景
     - 4.1.2 PHY DTS 配置
     - 4.1.3 Panel DTS 配置
       - 4.1.3.1 使用 EDID DTS 配置
       - 4.1.3.2 固定 Timing DTS 配置
         - 4.1.3.2.1 通过 EDID 回写 Timing
         - 4.1.3.2.2 根据 Datasheet 计算
   - 4.2 功能配置
     - 4.2.1 Panel 自测模式
     - 4.2.2 Controller 自测模式
     - 4.2.3 PSR 模式
     - 4.2.4 Connector-split 模式
     - 4.2.5 Dual-channel 模式
5. 常用 DEBUG 方法
   - 5.1 查看 DRM Connector 状态
   - 5.2 查看 Controller/PHY 寄存器
   - 5.3 查看 VOP 状态
   - 5.4 查看 Clock 配置
   - 5.5 强制开关 eDP
   - 5.6 读写 DPCD
   - 5.7 调整 DRM Log 类别
6. FAQ
   - 6.1 带宽计算方法
   - 6.2 DCLK 精度问题
   - 6.3 AUX 报错分析及解决方法
     - 6.3.1 检查 PHY 配置
     - 6.3.2 检查 Panel 配置
     - 6.3.3 确认 HPD 状态及上电时序
     - 6.3.4 检查 AUX 硬件设计
   - 6.4 Link Training 报错分析及解决方法
     - 6.4.1 确认 Panel 协议支持
     - 6.4.2 检查 Main Link 硬件设计
     - 6.4.3 检查硬件上的屏线和接口是否合规
     - 6.4.4 确认 HPD 状态及上电时序
     - 6.4.5 修改预加重和电压摆幅配置
       - 6.4.5.1 修改起始配置
       - 6.4.5.2 添加设置打印
     - 6.4.6 降速尝试
     - 6.4.7 增加 CR/EQ 阶段的 Retry 次数
     - 6.4.8 增加复位 Panel 操作

6.4.9 尝试强制输出
6.5 显示异常（Training 成功）分析及解决方法
6.5.1 开启自测模式
6.5.2 确认 VOP 状态是否符合预期
6.5.3 确认 Clock 配置是否符合预期
6.6 如何优化信号质量
6.6.1 优化 PE/VS Level 配置
6.6.2 开启 SSC 功能

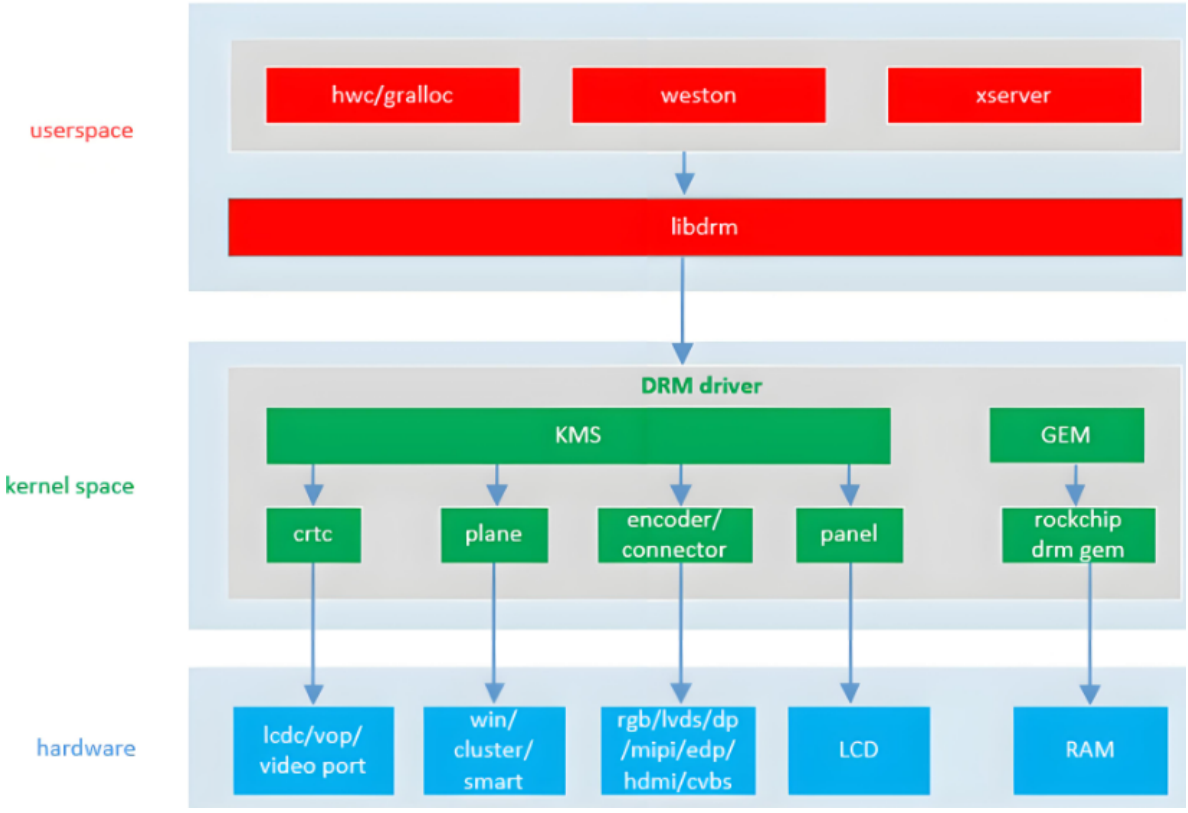
1. 概述

本文主要介绍 Rockchip 平台处理器 eDP 显示接口的基本特性、工作流程和常见问题的分析指南，目的是为了相关工程师能对 Rockchip 平台的 eDP 接口显示驱动框架和硬件接口有更好的理解，并通过常见问题的分析能快速定位问题、优化问题和解决问题。

2. eDP 基础介绍

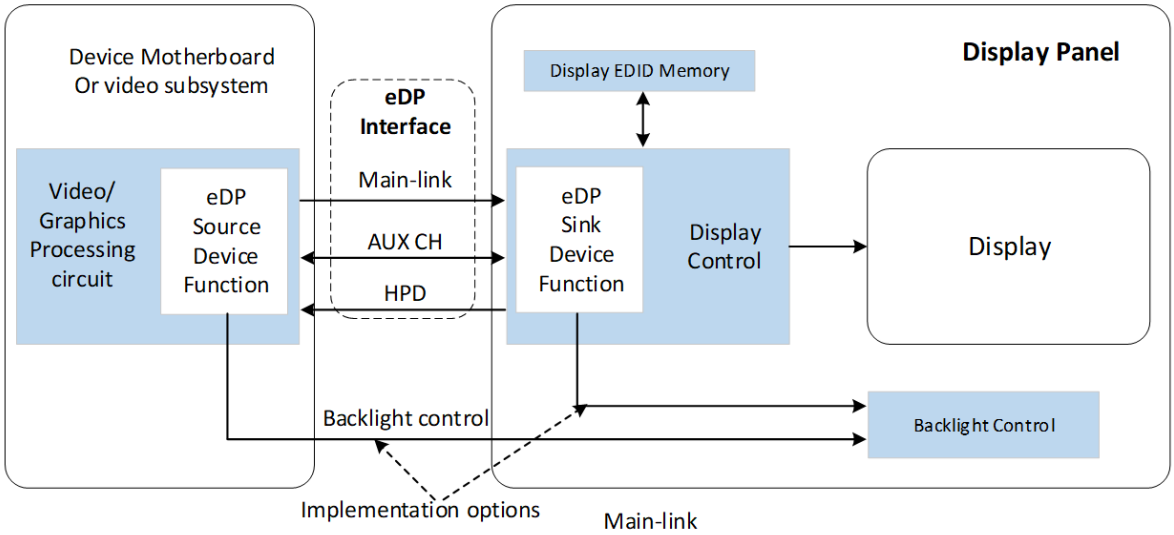
2.1 DRM 显示框架

应用通过 libdrm 和 DRM 驱动交互流程如下：



2.2 eDP 接口系统框架

下图为 eDP 典型的应用框图：

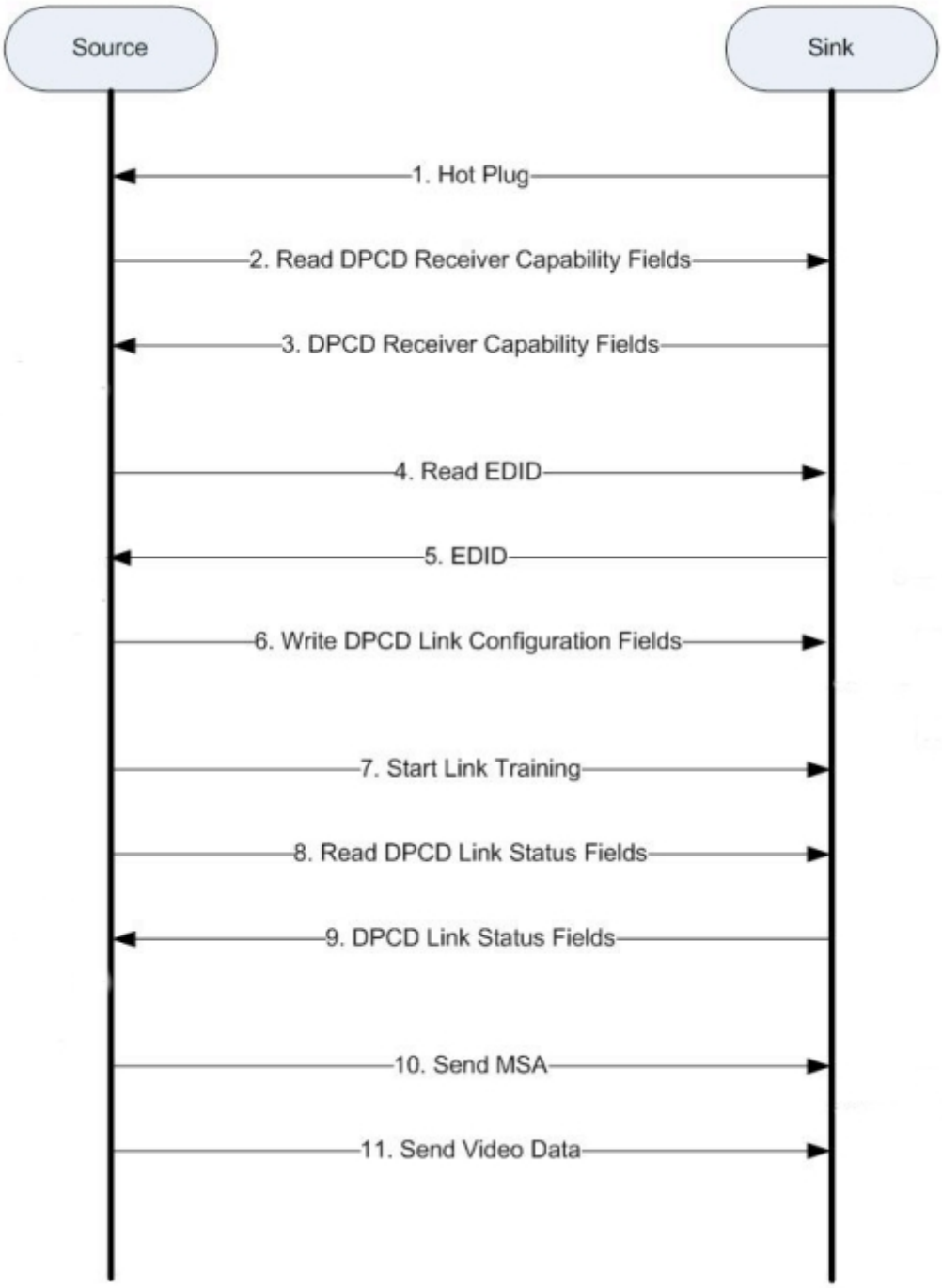


eDP 协议是基于 DP（Display Port）协议在嵌入式应用方向上的拓展，已广泛应用于笔记本电脑、平板电脑、手机等产品领域。

eDP 在硬件接口上主要由 Main-Link、AUX CH 与 HPD 三部分组成，共同实现 eDP Source 和 Sink 设备之间的交互功能 。

- Main-Link：由 1/2/4 对差分信号组成，用于传输视频和音频数据流
  - eDP v1.3 支持 RBR 1.62G/HBR 2.7G/HBR2 5.4G bps/lane，而 eDP v1.4 新增了 R216 2.16G/R243 2.43G/R324 3.24G/R432 4.32G bps/lane 四个 recommended 速率。
- AUX CH：为一对半双工的差分数据传输通路，用于 Source 和 Sink 设备之间的配置信息交互，如 DPCD 读写和 EDID 解析等，传输速率为 1 Mbps。
- HPD：在 Source 端通常为单个支持热插拔检测功能的 pin 脚，用于检测 Sink 设备的连接状态并响应其 HDP IRQ 请求。
  - Sink 设备从 disconnected 状态变为 connnected 状态，HDP 引脚应从低电平变为高电平。
  - HPD 可以反映 Sink 设备的工作状态，当 HPD 为高电平时通常表示 Sink 已处于可以正常显示的就绪状态，建议在硬件设计阶段连接 Source 和 Sink 两端的 HPD pin。

2.3 eDP Link Training



如上图所示，**Link Training** 的流程：

1：Sink 上电后向 Source 发出 HPD Plug 脉冲信号；

2-3：Source 读取 Sink DPCD 以确定 Sink 的 Features 支持情况；

4-5：Source 读取 Sink EDID 以确定 Sink 支持的显示时序信息；

6：Source 将 Link 配置写入到 DPCD 中，比如典型的 MAX_LINK_RATE 和 MAX_LANE_COUNT；

7：Source 发起 Link Training，Sink 进行 Clock Recovery 和 Equalization；

8-9：Source 读取 DPCD 获取 Link Status 以确定 Link Training 的结果，若失败则回退到步骤 7；

10：Source 在 Link Training 成功后发送 MSA 数据，Sink 解析视频流配置并恢复 Stream Clock；

11：Source 传输视频数据流。

3. Rockchip 平台 eDP 简介

3.1 功能特性

功能	RK3288	RK3399	RK3568	RK3588	RK3576
eDP Version	v1.3	v1.3	v1.3	v1.3	v1.3
DP Mode	Not support	Not support	Not support	v1.2	v1.2
Max Resolution	2560x1600@60Hz	2560x1600@60Hz	2560x1600@60Hz	3840x2160@60hz	3840x2160@60hz
Main-Link Lanes	1/2/4 lanes	1/2/4 lanes	1/2/4 lanes	1/2/4 lanes	1/2/4 lanes
Main-Link Rate	2.7/1.62 Gbps/lane	2.7/1.62 Gbps/lane	2.7/1.62 Gbps/lane	5.4/4.32/3.24/2.7/2.43/2.16/1.62 Gbps/lane	5.4/4.32/3.24/2.7/2.43/2.16/1.62 Gbps/lane
AUX_CH	1 M	1 M	1 M	1 M	1 M
Color Format	RGB	RGB	RGB	RGB/YUV444/YUV422	RGB/YUV444/YUV422
Color Depth	8 bpc	8 bpc	8 bpc	8/10 bpc (6bpc just for RGB)	8/10 bit (6bpc just for RGB)
ASSR	Support	Support	Support	Support	Support
I2S	Not support	Not support	Not support	Support	Support
SPDIF	Not support	Not support	Not support	Support	Support
PSR	Not support	Not support	Not support	Support	Support
Display Split Mode	Not support	Not support	Not support	Support	Support

3.2 eDP 与 VOP 连接关系

详见文档《Rockchip_Developer_Guide_DRM_Display_Driver_CN》的《VOP2 平台显示通路》章节。

3.3 驱动文件

U-Boot 驱动：

```
/* Controller */
drivers/video/drm/analogix_dp.c
drivers/video/drm/analogix_dp.h
drivers/video/drm/analogix_dp_reg.c

/* PHY */
/* RK3568 */
drivers/phy/phy-rockchip-naneng-edp.c
/* RK3588/RK3576 */
drivers/phy/phy-rockchip-samsung-hdptx.c

/* Panel */
drivers/video/drm/rockchip_panel.c
```

Kernel 驱动：

```
/* Controller */
drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
drivers/gpu/drm/bridge/analogix/analogix_dp_core.h
```

```
drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
drivers/gpu/drm/bridge/analogix/analogix_dp_reg.h
drivers/gpu/drm/rockchip/analogix_dp-rockchip.c
include/drm/bridge/analogix_dp.h

/* PHY */
/* RK3568 */
drivers/phy/rockchip/phy-rockchip-naneng-edp.c
/* RK3588/RK3576 */
drivers/phy/rockchip/phy-rockchip-samsung-hdptx.c

/* Panel */
drivers/gpu/drm/panel/panel-simple.c
```

- RK3288/RK3399 eDP PHY 无独立的驱动，Analogix DP Controller 驱动中包含 PHY 相关配置操作。

4. 软件配置

4.1 模块使能

4.1.1 Controller DTS 配置

eDP 控制器 DTS 配置取决于所连显示设备类型，需区分 Panel（屏幕）与 Monitor（显示器）两种场景，下文中将以 RK3588 平台 eDP0 输出为例说明。

4.1.1.1 HPD 配置

虽然 eDP 协议上说明 HPD 功能为 Optional，但 HPD 的电平高低是判断 Sink 设备工作状态的重要依据，建议在硬件设计阶段就考虑 HPD 功能的配置

- 若屏端出现异常，通过 HPD 可以通知到主控从而避免问题产生
- 配置 HPD 能有效减少由于时序配置不合理引起的显示异常问题，如 AUX 报错、Link Training 失败等

若硬件设计上不支持 HDP 功能，必须使能 Force HPD 配置；对于需要 HDP 的设计，Rockchip 平台可以支持 Function Pin HPD 和 GPIO HPD 两种方式：

- Force HPD

```
&edp0 {
    force-hpd;
};
```

- Function Pin HPD：

```
&edp0 {
    pinctrl-names = "default";
    pinctrl-0 = <&hdmim0_tx0_hpd>;
};
```

各个平台的 HPD Function Pin pinctrl 配置如下，需根据具体硬件设计原理图确认。

	Pinctrl 配置
RK3288	edp_hpd (GPIO7_B3)
RK3399	edp_hpd (GPIO4_C7)
RK3568	edpdpm0_pins (GPIO4_C4) | edpdpm1_pins (GPIO0_C2)
RK3588	hdmim0_tx0_hpd (GPIO1_A5) | hdmim0_tx1_hpd (GPIO1_A6) | hdmim1_tx0_hpd (GPIO3_D4) | hdmim1_tx1_hpd (GPIO3_B7)
RK3576	edp_txm0_pins (GPIO4_C1) | edp_txm1_pins (GPIO0_B6)

- GPIO HPD：

  可以选择任意 GPIO 来实现 HPD 功能。

```
&edp0 {
    pinctrl-names = "default";
    pinctrl-0 = <&edp0_hpd>;
    hpd-gpios = <&gpio1 RK_PA5 GPIO_ACTIVE_HIGH>;
};

&pinctrl {
    edp {
        edp0_hpd: edp0-hpd {
            rockchip,pins = <1 RK_PA5 0 &pcfg_pull_none>;
        };
    };
};
```

  在 Source 和 Sink 设备未通过线缆连接的情况下，若出现驱动误识别为 Connected 状态的异常问题，可将相应 GPIO 配置为下拉以提高稳定性。

```
&pinctrl {
    edp {
        edp0_hpd: edp0-hpd {
            rockchip,pins = <1 RK_PA5 0 &pcfg_pull_down>;
        };
    };
};
```

4.1.1.2 Panel 场景

```
&edp0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&hdmim0_tx0_hpd>;

    ports {
        port@1 {
            reg = <1>;
```

```
            edp_out_panel: endpoint {
                remote-endpoint = <&panel_in_edp>;
            };
        };
    };
};
```

4.1.1.3 Monitor 场景

RK3588/RK3576 平台的 eDP 接口可以支持 DP mode，作为 DP 使用时需要配置 HPD（详见《HPD DTS 配置》）。

参考 DTS arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4-v10-hdmi2dp.dts（命名缘由：由于 HDMI 和 eDP PHY 上是 combo 的，该板子直接将 HDMI 母座转成 DP 母座以将 eDP 作为 DP 使用）：

```
&edp1 {
    pinctrl-names = "default";
    pinctrl-0 = <&hdmim0_tx1_hpd>;
    status = "okay";
};
```

4.1.2 PHY DTS 配置

RK3568/RK3588/RK3576 需要配置 PHY DTS，而 RK3288/RK3399 不需要。

```
/* RK3568 */
&edp_phy {
        status = "okay";
};

/* RK3588/RK3576 */
&hdptxphy0 {
       status = "okay";
};

&hdptxphy_hdmi0 {
       status = "disabled";
};
```

- 在 RK3588 和 RK3576 上，eDP 和 HDMI 接口在 PHY 上是 Combo 的，在使用 eDP 的时候，要确保相应的 HDMI PHY 节点 hdptxphy_hdmi0/hdptxphy_hdmi1 是 disabled。
- RK3588 和 RK3576 上，eDP PHY 可以支持配置 Link Training Table 以替换默认值，用于优化信号质量和调整眼图指标等场景。

  ```
  &hdptxphy0 {
          /* Single Vdiff Training Table (optional) */
          training-table = /bits/ 8 <
              /* voltage swing 0, pre-emphasis 0->3 */
              0x0d 0x0a 0x04 0x06 0x00 0x04
              0x0d 0x0a 0x04 0x06 0x00 0x04
              0x0d 0x0a 0x04 0x06 0x00 0x04
              0x0d 0x0a 0x04 0x06 0x00 0x04
  ```

```
                /* voltage swing 1, pre-emphasis 0->2 */
                0x0d 0x0a 0x04 0x06 0x00 0x04
                0x0d 0x0a 0x04 0x06 0x00 0x04
                0x0d 0x0a 0x04 0x06 0x00 0x04
                /* voltage swing 2, pre-emphasis 0->1 */
                0x0d 0x0a 0x04 0x06 0x00 0x04
                0x0d 0x0a 0x04 0x06 0x00 0x04
                /* voltage swing 3, pre-emphasis 0 */
                0x0d 0x0a 0x04 0x06 0x00 0x04
        >;
        status = "okay";
};
```

- RK3588 和 RK3576 eDP 支持 Lane 差分信号 P/N Swap 功能，可以用于硬件设计上 Source 和 Sink 间 Lane P/N 信号接反的场景，参考配置如下：

```
&hdptxphy0 {
        /* swap lane1 P/N */
        lane-polarity-invert = <0 1 0 0>;
        status = "okay";
};
```

4.1.3 Panel DTS 配置

Panel 场景下 eDP 的显示时序是通过 Panel 驱动传递的，Rockchip 平台 eDP Panel 使用的驱动为 drivers/gpu/drm/panel/panel-simple.c。

- Kernel-5.10 及以上参考文档为 Documentation/devicetree/bindings/display/panel/panel-simple.yaml。
- Kernel-4.19 及以下参考文档为 Documentation/devicetree/bindings/display/panel/simple-panel.txt。

eDP 需要输出 Panel 的 Timing 才能正常显示，Timing 相关可参考《Rockchip_Developer_Guide_DRM_Display_Driver_CN.pdf》的 ”扫描时序说明“ 章节。

获取 Panel Timing 的方式有：

- 通过读取 EDID 的方式获取（推荐方式）
- 通过 Panel Datasheet 中的 Singal Timing Specifications 表格获取
- 联系屏幕厂家获取 Panel Timing

注意：对于部分屏幕，EDID 有可能损坏读不到，出现这种情况才建议从其他方式获取 Timing

DTS 配置有以下两种方式：

- 不配置 Timing 节点，通过 EDID 获取 Timing 并输出（推荐配置）
- 配置 Timing 节点，通过配置的固定 Timing 输出

HPD 及 Delay 要求：

- 若使用不配置 Timing 节点的方式，硬件设计上要连接 HPD 以防止时序异常导致的问题
- 对于配置 Timing 节点的方式，也推荐连接 HPD
    - 若不连接 HPD，则必须要配置 prepare-delay-ms （控制 VCC 上电后到首次 AUX 访问的间隔时间）以满足 Panel 上电时序要求

4.1.3.1 使用 EDID DTS 配置

以 RK3588S EVB1 为例，Kernel-6.1 相应 DTS 为 arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi。

```
panel-edp {
    compatible = "simple-panel";
    backlight = <&backlight>;
    power-supply = <&vcc3v3_lcd_edp>;
    prepare-delay-ms = <120>;
    enable-delay-ms = <120>;
    unprepare-delay-ms = <120>;
    disable-delay-ms = <120>;
    width-mm = <120>;
    height-mm = <160>;

    port {
        panel_in_edp: endpoint {
            remote-endpoint = <&edp_out_panel>;
        };
    };
};
```

- backlight 节点的 pwms 配置需要根据硬件实际的连接情况修改，在显示图像前需要确保背光已经正常点亮。
  - 详见 PWM 模块参考文档《Rockchip_Developer_Guide_Linux_PWM_CN》。
- power-supply 配置的是 Panel 的供电
  - 对于 Sink 异常的场景，强行对 Panel 或者 Bridge 进行下上电去重置设备是常用手段，要确保 Panel 供电可由主控控制上下电。
- enable-gpios/reset-gpios 用于 Panel 的使能/重置引脚配置，需要根据实际硬件电路设计来修改。
  - （可选）enable 引脚通常用于屏端供电的使能，gpio 配置取决于供电电路的具体设计。
  - （可选）reset 引脚通常屏端会直接引出，并在 datasheet 中说明触发复位功能的条件，gpio 配置取决于复位电路的具体设计。
- delay-ms 配置需要满足 Panel Datasheet 中上下电和复位的时序要求
  - （可选）enable-delay-ms/disable-delay-ms 用于 enable 引脚的使能后或关闭前的延时控制
  - （可选）reset-delay-ms 用于 reset 引脚使能后的延时控制
  - （可选）prepare-delay-ms/unprepare-delay-ms 用于 power-supply 电源使能后或关闭前的延时控制

（以下关于 delay-ms 的说明摘自 Documentation/devicetree/bindings/display/panel/panel-simple.yaml）

```
......
  prepare-delay-ms:
    description: |
      the time (in milliseconds) that it takes for the panel to
      become ready and start receiving video data
  enable-delay-ms:
    description: |
      the time (in milliseconds) that it takes for the panel to
      display the first valid frame after starting to receive
      video data
  disable-delay-ms:
    description: |
      the time (in milliseconds) that it takes for the panel to
      turn the display off (no content is visible)
  unprepare-delay-ms:
    description: |
```

```
        the time (in milliseconds) that it takes for the panel
        to power itself down completely
    reset-delay-ms:
      description: |
        the time (in milliseconds) that it takes for the panel to
        reset itself completely
    init-delay-ms:
      description: |
        the time (in milliseconds) that it takes for the panel to
        send init command sequence after reset deassert
  ......
```

4.1.3.2 固定 Timing DTS 配置

DTS 固定 Panel Timing 优点：

- 接口不再去读取 EDID 信息，可以避免因为读取 EDID 出错造成的异常情况
- 可以根据需求调整 Timing 参数，可以避免 Pixel 需要对齐或 Clock 需要特殊分频造成的问题

DTS 固定 Panel Timing 缺点：

- Timing 配置不符合 Panel 需求，会导致 Training 失败或异常显示

对于某些 Panel 的特殊情况，如 EDID 数据错乱、Clock 频率特殊、Timing 参数需要调整等，我们可以固定 Timing DTS 配置。

DRM 框架对于 display_timing 结构体及其变量的描述详见 include/video/display_timing.h，示例如下：

```
/*
 * Single "mode" entry. This describes one set of signal timings a display can
 * have in one setting. This struct can later be converted to struct videomode
 * (see include/video/videomode.h). As each timing_entry can be defined as a
 * range, one struct display_timing may become multiple struct videomodes.
 *
 * Example: hsync active high, vsync active low
 *
 *                  Active Video
 * Video  ___________________________XXXXXXXXXXXXXXXXXXXXXX_____________________
 *    |<- sync ->|<- back ->|<----- active ----->|<- front ->|<- sync..
 *    |      |   porch  |            |   porch   |
 *
 * HSync _|__________|_____________________________________________|_________
 *
 * VSync __|__________|_____________________________________________|_________
 */
struct display_timing {
    struct timing_entry pixelclock;

    struct timing_entry hactive;        /* hor. active video */
    struct timing_entry hfront_porch;   /* hor. front porch */
    struct timing_entry hback_porch;    /* hor. back porch */
    struct timing_entry hsync_len;      /* hor. sync len */

    struct timing_entry vactive;        /* ver. active video */
    struct timing_entry vfront_porch;   /* ver. front porch */
    struct timing_entry vback_porch;    /* ver. back porch */
```

```
    struct timing_entry vsync_len;      /* ver. sync len */

    enum display_flags flags;       /* display flags */
};
```

关于 **Timing** 基础参数的计算方式参考以下：

- htotal = hsync_len + hback_porch + hactive + hfront_porch
- hsync_start = hfront_porch + hactive
- hsync_end = htotal - hback_porch = hsync_len + hactive + hfront_porch
- vtotal = vsync_len + vback_porch + vactive + vfront_porch
- vsync_start = vfront_porch + vactive
- vsync_end = vtotal - vback_porch = vsync_len + vactive + vfront_porch

获取 **Timing** 后配置在 **Panel** 节点内，**Timing** 参数会被 **DRM** 框架识别装填到对应的 **display_timing** 结构体中，配置参考如下：

```
panel-edp {
    compatible = "simple-panel";
    backlight = <&backlight>;
    power-supply = <&vcc3v3_lcd_edp>;
    prepare-delay-ms = <120>;
    enable-delay-ms = <120>;
    unprepare-delay-ms = <120>;
    disable-delay-ms = <120>;
    width-mm = <120>;
    height-mm = <160>;

    panel-timing {
        clock-frequency = <200000000>;
        hactive = <1536>;
        vactive = <2048>;
        hfront-porch = <12>;
        hsync-len = <16>;
        hback-porch = <48>;
        vfront-porch = <8>;
        vsync-len = <4>;
        vback-porch = <8>;
        hsync-active = <0>;
        vsync-active = <0>;
        de-active = <0>;
        pixelclk-active = <0>;
    };

    port {
        panel_in_edp: endpoint {
            remote-endpoint = <&edp_out_panel>;
        };
    };
};
```

- clock-frequency 属性值会决定 VOP 输出的帧率，计算中需要用到 DRM 框架的时序属性 htotal 和 vtotal。
    - 对于 clock-frequency 过于特殊无法分到的情况，可以尝试抹零数值保留到单位 MHz 进行测试。

- 帧率的计算则可以参考 drivers/gpu/drm/drm_modes.c 中 drm_mode_vrefresh() 函数的实现，设帧率为 fr 则计算公式为：

$$fr = \frac{clock}{htotal \times vtotal}$$

```
/**
 * drm_mode_vrefresh - get the vrefresh of a mode
 * @mode: mode
 *
 * Returns:
 * @modes's vrefresh rate in Hz, rounded to the nearest integer.
Calculates the
 * value first if it is not yet set.
 */
int drm_mode_vrefresh(const struct drm_display_mode *mode)
{
    unsigned int num, den;

    if (mode->htotal == 0 || mode->vtotal == 0)
        return 0;

    num = mode->clock;
    den = mode->htotal * mode->vtotal;

    if (mode->flags & DRM_MODE_FLAG_INTERLACE)
        num *= 2;
    if (mode->flags & DRM_MODE_FLAG_DBLSCAN)
        den *= 2;
    if (mode->vscan > 1)
        den *= mode->vscan;

    return DIV_ROUND_CLOSEST_ULL(mul_u32_u32(num, 1000), den);
}
EXPORT_SYMBOL(drm_mode_vrefresh);
```

- hsync-active/vsync-active/de-active/pixelclk-active 各属性会决定同步信号 hsync/vsync/den 及 pixelclk 的极性（宏定义参考下文 display_flags 部分）
  - `hsync-active = <0>;`
    - 表示 DISPLAY_FLAGS_HSYNC_LOW，即 HSYNC 信号为低有效；反之为 DISPLAY_FLAGS_HSYNC_HIGH，HSYNC 信号为高有效。
  - `vsync-active = <0>;`
    - 表示 DISPLAY_FLAGS_VSYNC_LOW，即 VSYNC 信号为低有效；反之为 DISPLAY_FLAGS_VSYNC_HIGH，VSYNC 信号为高有效。
  - `de-active = <0>;`
    - 表示 DISPLAY_FLAGS_DE_LOW，即 DEN 信号为低有效；反之为 DISPLAY_FLAGS_DE_HIGH，DEN 信号为高有效。
  - `pixelclk-active = <0>;`
    - 表示 DISPLAY_FLAGS_PIXDATA_NEGEDGE，即 DATA 信号在 Clock 下降沿有效；反之为 DISPLAY_FLAGS_PIXDATA_POSEDGE，即 DATA 信号在 Clock 上升沿有效。

4.1.3.2.1 通过 EDID 回写 Timing

只要 DTS Panel 节点中没有配置 Timing 节点（参考 4.1.3.1 使用 EDID DTS 配置示例），默认就会使用 AUX 通道去读取 Panel EDID 信息

接入屏幕后会在 U-Boot 串口中打印类似下面的信息：

```
#老版本代码会打印 EDID 其他信息
Maximum visible display size: 26 cm x 17 cm
Power management features: no active off, no suspend, no standby
Estabilished timings:
Standard timings:
2400x1600 59 Hz (detailed)
Monitor name: LQ123P1JX31
Detailed mode clock 252750 kHz, flags[a]
H: 2400 2448 2480 2560
V: 1600 1603 1613 1646
bus_format: 1009
```

```
#新版本不会打印 EDID 其他信息
edp@fded0000:  detailed mode clock 148500 kHz, flags[5]
    H: 1920 2008 2052 2200
    V: 1080 1084 1089 1125
bus_format: 100a
```

重点确认 U-Boot Log 中没有出现以下语句，如有出现说明 DTS 对应的 Panel 节点还是配置了 Timing，接口就不会去读取 EDID 信息：

```
Using display timing dts
```

通过上面的信息我们可以获取到：

- clock-frequency = 252750 kHz
- hactive = 2400　hsync_start = 2448　hsync_end = 2480　htotal = 2560
- vactive = 1600　vsync_start = 1603　vsync_end = 1613　vtotal = 1646
- flags = 0xa
    - 对应的就是 Timing 节点中的 hsync-active/vsync-active/de-active/pixelclk-active 等 display_flags。
    - display_flags 的定义详见 include/video/display_timing.h：

```
enum display_flags {
    DISPLAY_FLAGS_HSYNC_LOW     = BIT(0),
    DISPLAY_FLAGS_HSYNC_HIGH    = BIT(1),
    DISPLAY_FLAGS_VSYNC_LOW     = BIT(2),
    DISPLAY_FLAGS_VSYNC_HIGH    = BIT(3),

    /* data enable flag */
    DISPLAY_FLAGS_DE_LOW        = BIT(4),
    DISPLAY_FLAGS_DE_HIGH       = BIT(5),
    /* drive data on pos. edge */
    DISPLAY_FLAGS_PIXDATA_POSEDGE   = BIT(6),
    /* drive data on neg. edge */
    DISPLAY_FLAGS_PIXDATA_NEGEDGE   = BIT(7),
    DISPLAY_FLAGS_INTERLACED    = BIT(8),
    DISPLAY_FLAGS_DOUBLESCAN    = BIT(9),
    DISPLAY_FLAGS_DOUBLECLK     = BIT(10),
    /* drive sync on pos. edge */
    DISPLAY_FLAGS_SYNC_POSEDGE  = BIT(11),
```

```
    /* drive sync on neg. edge */
    DISPLAY_FLAGS_SYNC_NEGEDGE  = BIT(12),
};
```

- bus_format = 1009
  - 指的是当前使用的色彩格式，详情参考 `include/uapi/linux/media-bus-format.h`

```
/* RGB - next is    0x1024 */
#define MEDIA_BUS_FMT_RGB444_1X12       0x1016
#define MEDIA_BUS_FMT_RGB444_2X8_PADHI_BE   0x1001
#define MEDIA_BUS_FMT_RGB444_2X8_PADHI_LE   0x1002
#define MEDIA_BUS_FMT_RGB555_2X8_PADHI_BE   0x1003
#define MEDIA_BUS_FMT_RGB555_2X8_PADHI_LE   0x1004
#define MEDIA_BUS_FMT_RGB565_1X16       0x1017
#define MEDIA_BUS_FMT_BGR565_2X8_BE     0x1005
#define MEDIA_BUS_FMT_BGR565_2X8_LE     0x1006
#define MEDIA_BUS_FMT_RGB565_2X8_BE     0x1007
#define MEDIA_BUS_FMT_RGB565_2X8_LE     0x1008
#define MEDIA_BUS_FMT_RGB666_1X18       0x1009
#define MEDIA_BUS_FMT_RBG888_1X24       0x100e
#define MEDIA_BUS_FMT_RGB666_1X24_CPADHI    0x1015
#define MEDIA_BUS_FMT_RGB666_1X7X3_SPWG     0x1010
#define MEDIA_BUS_FMT_BGR888_1X24       0x1013
#define MEDIA_BUS_FMT_BGR888_3X8        0x101b
#define MEDIA_BUS_FMT_GBR888_1X24       0x1014
#define MEDIA_BUS_FMT_RGB888_1X24       0x100a
#define MEDIA_BUS_FMT_RGB888_2X12_BE        0x100b
#define MEDIA_BUS_FMT_RGB888_2X12_LE        0x100c
#define MEDIA_BUS_FMT_RGB888_3X8        0x101c
#define MEDIA_BUS_FMT_RGB888_3X8_DELTA      0x101d
#define MEDIA_BUS_FMT_RGB888_1X7X4_SPWG     0x1011
#define MEDIA_BUS_FMT_RGB888_1X7X4_JEIDA    0x1012
```

4.1.3.2.2 根据 Datasheet 计算

下图截自示例 DTS Panel "lg,lp079qx1-sp0v" 对应的 Datasheet：

ITEM	Symbol		Min.	Typ.	Max.	Unit	Note
DCLK	Frequency	f_{CLK}	-	200.02	-	MHz	
Hsync	Active	t_{WHA}	-	1536	-	tCLK	
	Period	t_{HP}	-	1612	-		
	Width-Active	t_{WH}	-	16	-		
Vsync	Active	t_{WVA}	-	2048	-	tHP	
	Period	t_{VP}	-	2068	-		
	Width-Active	t_{WV}	-	4	-		
Data Enable	Horizontal back porch	t_{HBP}	-	48	-	tCLK	
	Horizontal front porch	t_{HFP}	-	12	-		
	Vertical back porch	t_{VBP}	-	8	-	tHP	
	Vertical front porch	t_{VFP}	-	8	-		

根据 Panel Datasheet 进行配置，用户可以根据具体的应用需求在 Datasheet 指定的上下阈值区间内微调。

将计算出的 Timing 参数参考上文固定在 eDP Panel Timing 节点，即可根据指定的 Timing 进行输出。

4.2 功能配置

4.2.1 Panel 自测模式

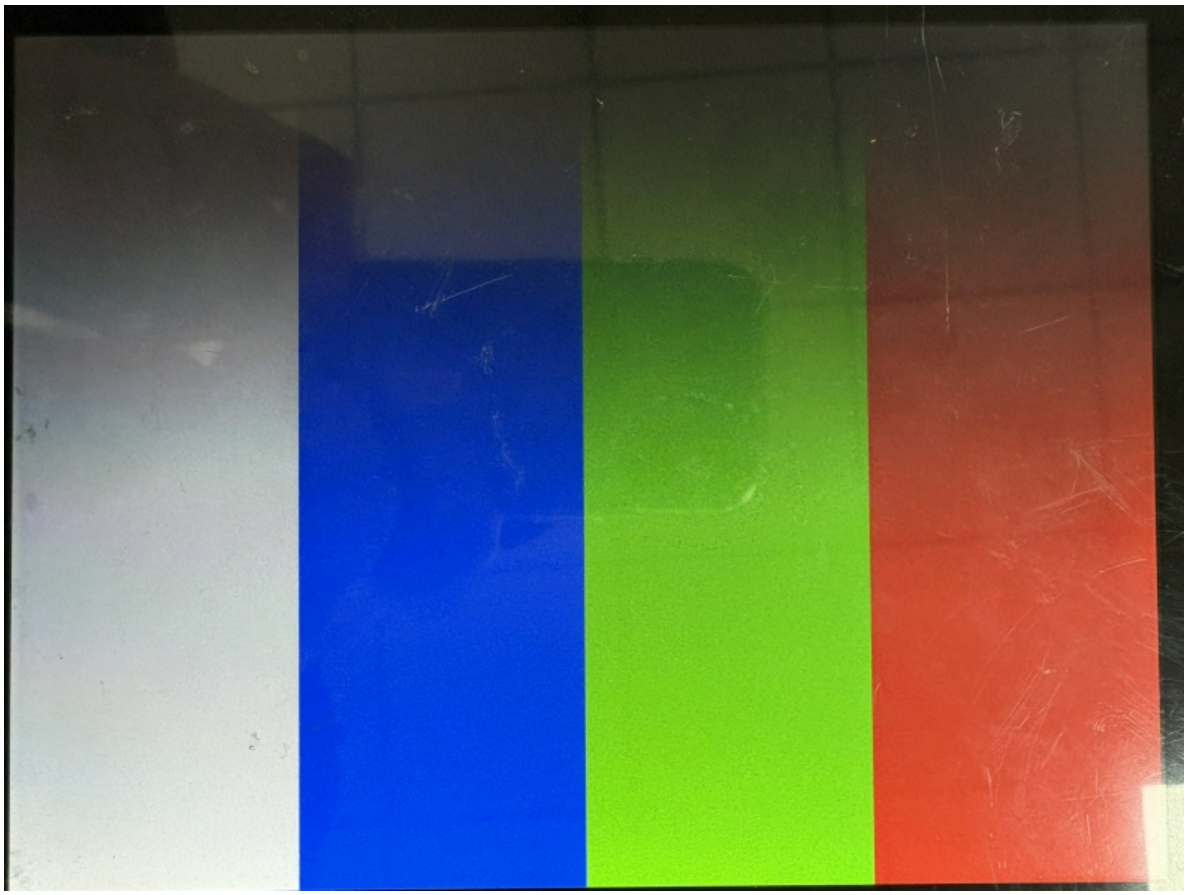
该模式通常用于 Panel 调试阶段，若 Panel 显示异常但开启 Panel 自测模式后显示正常，说明 Sink 端工作状态正常，需要从 Source 端确认问题，详见《常见问题及分析方法》。

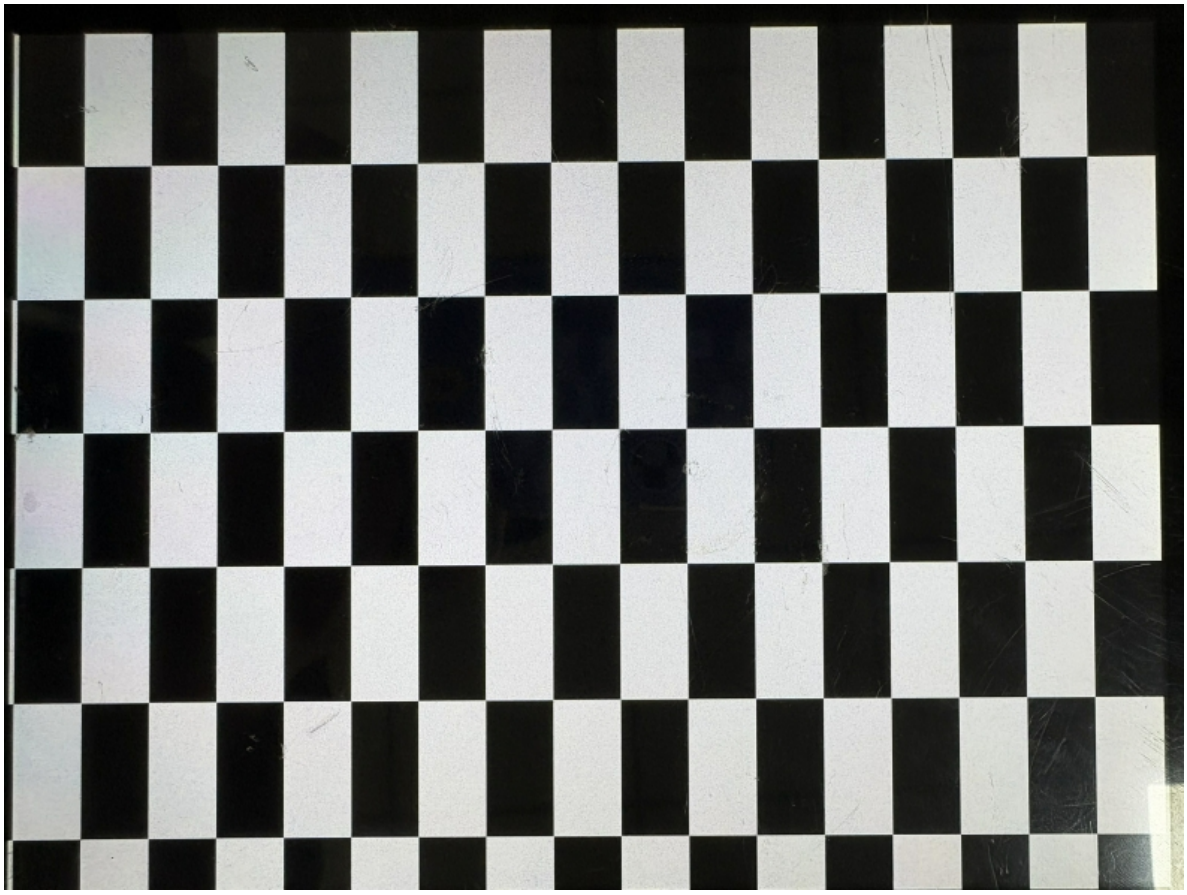
以 RK3588S EVB1 arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi 为例，DTS 中需要在 edp 节点加上 panel-self-test 属性：

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
index 8c1d0f37b8a3..e58dd8aae2ee 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
@@ -262,6 +262,7 @@ &dp0_sound{
 };

 &edp0 {
+       panel-self-test;
        force-hpd;
        status = "okay";
```

对于 "lg,lp079qx1-sp0v" 这块屏幕，Panel 自测模式为下述两张图片循环动态切换：





4.2.2 Controller 自测模式

该模式通常用于 Panel 调试阶段，若一款屏显示异常，但在开启 Controller 自测模式后显示正常，则说明 eDP Controller -> PHY -> Panel 这个显示链路是正常的，需要从显示链路更上游的 VOP 上定位分析，关于 VOP 的分析可以参考文档 《Rockchip_Developer_Guide_DRM_Display_Driver_CN》。

以 RK3588S EVB1 arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi 为例，DTS 中需要在 edp 节点加上 analogix,video-bist-enable 属性：

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
index 8c1d0f37b8a3..07a9de32f910 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
@@ -262,6 +262,7 @@ &dp0_sound{
 };

 &edp0 {
+       analogix,video-bist-enable;
        force-hpd;
        status = "okay";
```

eDP Controller 自测模式为固定显示下述图片：



4.2.3 PSR 模式

PSR（Panel Self Refresh）是 eDP v1.3 引入的 Source/Sink optional Feature，只有在 Source/Sink 端都支持该 Feature 时才能正常启用。

PSR 功能开启后，在静态显示场景下（比如 Android 静态桌面显示状态），VOP -> eDP -> Panel 显示链路上的 VOP 和 eDP 模块均会自动进入到低功耗状态，可以有效降低系统功耗。Kernel-6.1 DTS arch/arm64/boot/dts/rockchip/rk3576-evb1-v10-edp-NE160QAM-NX1.dts 配置下 eDP -> Panel（NE160QAM-NX1）单显静态桌面场景，在开启 PSR 功能后实测可以降低约 470mW 的平均系统功耗。

```
commit 303b0de1b8fd9f33a8cbfa935e0885c53fe9f78c
Author: Damon Ding <damon.ding@rock-chips.com>
Date:   Wed Jul 16 14:22:21 2025 +0800

    drm/bridge: analogix_dp: Add &analogix_dp_plat_data.disable_psr to check
whether to disable PSR

    First of all, since the PSR feature can help to reduce the power
    consumption, the Source device, which can support PSR function,
    should enable PSR if the PSR capability of Sink device is detected
    rather than depending on the user to add 'support-psr' DTS property
    manually.

    Different platforms that use the same Analogix DP bridge driver may
    have different methods for parsing the PSR capability. Therefore, add
    a new flag &analogix_dp_plat_data.disable_psr to disable PSR forcely,
    which set in the platform side, should be more reasonable.

    If the user truly does not want to enable PSR function or the Panel
```

```
    has something wrong with it, the property 'rockchip,disable-psr' will
    be helpful.

    Fixes: 9622f2d0f10a ("drm/bridge: analogix_dp: disable PSR feature by
default")
    Change-Id: Id2fce34857df80de5a1ec97f342709a6e2840ed4
    Signed-off-by: Damon Ding <damon.ding@rock-chips.com>
```

对于包含上述提交的 Kernel 版本，eDP 接口驱动会通过 DPCD（DP_PSR_SUPPORT 0x70）来确认 Sink 端的 PSR 功能支持情况，若 Sink 端支持该功能将自动使能 Source 端的 PSR Feature。若遇到 Sink 端 DPCD DP_PSR_SUPPORT 为 1 而 PSR 功能支持异常的情况，建议与屏厂进一步确认 PSR 功能实际的支持情况，并可以通过配置 edp 节点下的 rockchip,disable-psr 属性强制关闭 PSR 功能：

```
&edp {
    rockchip,disable-psr;
};
```

而未包含该提交的版本，需要通过配置 edp 节点下的 support-psr 属性来手动使能 PSR 功能：

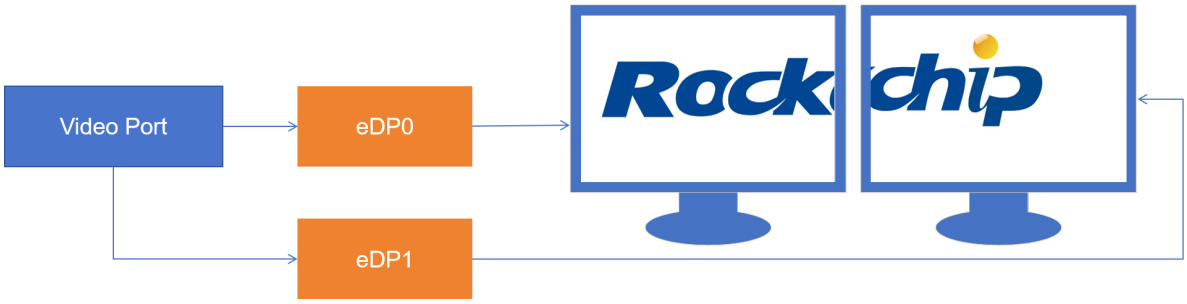
```
&edp {
    support-psr;
};
```

4.2.4 Connector-split 模式

VOP2 提供的 Connector-split 功能是一种类似 MIPI Dual Channel Mode 的技术，可以让单个 Video Port 的输出在水平方向上一分为二，以驱动两个时序一致的相同/不同接口（详见文档《Rockchip_Developer_Guide_DRM_Display_Driver_CN》的《Connector-spilt》章节）。

此处以双 eDP Connector-split 场景为例说明，应用场景上为：

1. 单个 VP 驱动两个时序一致的 eDP 屏。
2. 两个 eDP 屏分别显示左右半屏图像。
3. DRM 框架仅注册一个 Connector，应用层仅能查询到 eDP0 设备。



示例 DTS 配置为 Kernel-6.1 arch/arm64/boot/dts/rockchip/rk3588-vehicle-serdes-display-v20.dtsi，关键配置如下：

- edp0 和 edp1 需要在同一个 VP 输出，对应实例中同时使能 edp0_in_vp1 和 edp1_in_vp1。
- Connector-split 功能中左侧显示的接口需要加上 rockchip,split-mode 属性。

```
&edp0 {
    rockchip,split-mode;
    force-hpd;
    status = "okay";
```

```
    ports {
        port@1 {
            reg = <1>;

            edp0_out_i2c5_bu18tl82: endpoint {
                remote-endpoint = <&i2c5_bu18tl82_in_edp0>;
            };
        };
    };
};

&edp0_in_vp0 {
    status = "disabled";
};

&edp0_in_vp1 {
    status = "okay";
};

&edp0_in_vp2 {
    status = "disabled";
};

&edp1 {
    force-hpd;
    status = "okay";

    ports {
        port@1 {
            reg = <1>;

            edp1_out_i2c7_bu18tl82: endpoint {
                remote-endpoint = <&i2c7_bu18tl82_in_edp1>;
            };
        };
    };
};

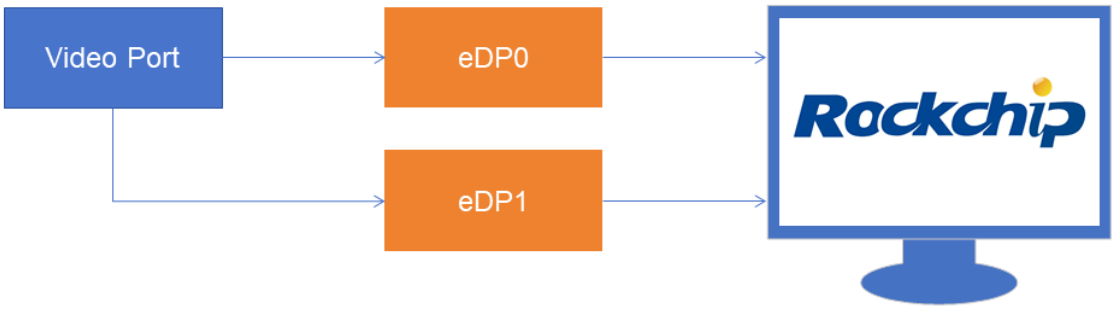
&edp1_in_vp0 {
    status = "disabled";
};

&edp1_in_vp1 {
    status = "okay";
};

&edp1_in_vp2 {
    status = "disabled";
};
```

4.2.5 Dual-channel 模式

Connector-split 功能单个 VP 驱动的两个接口通常对应两个屏幕/显示器，而对于两个同类接口共同驱动单个屏幕/显示器的场景，则需要打开左侧显示接口的 rockchip,dual-channel 属性。



Dual-channel 模式可以理解为 Connector-split 功能的一个子集，在实现机制上与其相同，差异在于左侧 connector 对应时序需要配置为 VP 实际输出的时序，而使能 split-mode 属性时配置的是 VP 实际时序的 1/2 。

以一款 8 lane 3840x2160@144Hz eDP panel 模组 M280DCA 为例，datasheet 中要求的 data mapping 如下：

Area	Lane	Data Stream
Left	Lane 0	Pixel 1,5,9, ... ,1913,1917
	Lane 1	Pixel 2,6,10, ... ,1914,1918
	Lane 2	Pixel 3,7,11, ... ,1915,1919
	Lane 3	Pixel 4,8,12, ... ,1916,1920
Right	Lane 4	Pixel 1921,1925,1929, ... ,3833,3837
	Lane 5	Pixel 1922,1926,1930, ... ,3834,3838
	Lane 6	Pixel 1923,1927,1931, ... ,3835,3839
	Lane 7	Pixel 1924,1928,1932, ... ,3836,3840

RK3588 单 eDP 支持 4 lane 3840x2160@60Hz，8 lane 3840x2160@144Hz 则可以通过 Dual-channel 功能来实现，示例 DTS 为 arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4-v10-edp-8lanes-M280DCA.dts，关键配置如下：

- edp0 和 edp1 需要在同一个 VP 输出，对应实例中同时使能 edp0_in_vp0 和 edp1_in_vp0。
- Dual-channel 功能中左侧显示的接口需要加上 rockchip,dual-channel 属性。

```
/ {
    panel-edp {
        compatible = "simple-panel";
        backlight = <&backlight>;
        power-supply = <&vcc3v3_edp>;
        enable-gpios = <&gpio4 RK_PC1 GPIO_ACTIVE_HIGH>;
        prepare-delay-ms = <120>;
        enable-delay-ms = <120>;
        unprepare-delay-ms = <120>;
        disable-delay-ms = <120>;

        display-timings {
            native-mode = <&timing_4kp144>;
            timing_4kp144: timing0 {
                clock-frequency = <1360800000>;
```

```
                hactive = <3840>;
                vactive = <2160>;
                hfront-porch = <160>;
                hsync-len = <40>;
                hback-porch = <160>;
                vfront-porch = <40>;
                vsync-len = <10>;
                vback-porch = <40>;
                hsync-active = <0>;
                vsync-active = <0>;
                de-active = <0>;
                pixelclk-active = <0>;
            };

            timing_4kp120: timing1 {
                clock-frequency = <1188000000>;
                hactive = <3840>;
                vactive = <2160>;
                hfront-porch = <240>;
                hsync-len = <80>;
                hback-porch = <240>;
                vfront-porch = <40>;
                vsync-len = <10>;
                vback-porch = <40>;
                hsync-active = <0>;
                vsync-active = <0>;
                de-active = <0>;
                pixelclk-active = <0>;
            };
        };

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                panel_in_edp0: endpoint {
                    remote-endpoint = <&edp0_out_panel>;
                };
            };

            port@1 {
                reg = <1>;
                panel_in_edp1: endpoint {
                    remote-endpoint = <&edp1_out_panel>;
                };
            };
        };
    };
};

&edp0 {
    force-hpd;
    status = "okay";
    dual-channel;

    ports {
```

```
        port@1 {
            reg = <1>;

            edp0_out_panel: endpoint {
                remote-endpoint = <&panel_in_edp>;
            };
        };
    };
};

&edp0 {
    force-hpd;
    rockchip,dual-channel;
    rockchip,data-swap;
    status = "okay";

    ports {
        port@1 {
            reg = <1>;

            edp0_out_panel: endpoint {
                remote-endpoint = <&panel_in_edp0>;
            };
        };
    };
};

&edp0_in_vp0 {
    status = "okay";
};

&edp0_in_vp1 {
    status = "disabled";
};

&edp0_in_vp2 {
    status = "disabled";
};

&edp1 {
    force-hpd;
    status = "okay";

    ports {
        port@1 {
            reg = <1>;

            edp1_out_panel: endpoint {
                remote-endpoint = <&panel_in_edp1>;
            };
        };
    };
};

&edp1_in_vp0 {
    status = "okay";
};
```

```
&edp1_in_vp1 {
    status = "disabled";
};

&edp1_in_vp2 {
    status = "disabled";
};
```

5. 常用 DEBUG 方法

5.1 查看 DRM Connector 状态

以 RK3588S EVB1 为例说明，相应 DTS arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi。

在 Linux 系统下，在 /sys/class/drm 目录下可以看到已注册的各个 DRM 设备。如下 logs 所示，card0 对于 Rockchip 平台表示 Display-subsystem 设备，而card0-DP-1 和 card0-eDP-1 对应 DP 接口设备和 eDP 接口设备。

```
console:/ # cat /sys/class/drm/
card0-DP-1/         card0-eDP-1/        card1/              renderD129/
card0-Writeback-1/  card0/              renderD128/         version
```

以 card0-eDP-1 为例，其目录下有如下内容：

```
console:/ # ls /sys/class/drm/card0-eDP-1/
device  dpms  edid  enabled  modes  power  status  subsystem  uevent
```

- dpms：设备的 Power Status。

  ```
  console:/ # cat /sys/class/drm/card0-eDP-1/dpms
  On
  ```

- edid：从 Sink 设备获取的 EDID 信息.
  - 可以通过 edid-decode 指令来解析：

    ```
    console:/ # edid-decode /sys/class/drm/card0-eDP-1/edid
    edid-decode (hex):

    00 ff ff ff ff ff ff 00 16 83 00 00 00 00 00 00
    04 17 01 00 a5 10 0c 78 06 ef 05 a3 54 4c 99 26
    0f 50 54 00 00 00 01 01 01 01 01 01 01 01 01 01
    01 01 01 01 01 01 ea 4e 00 4c 60 00 14 80 0c 10
    84 00 78 a0 00 00 00 18 00 00 00 10 00 00 00 00
    00 00 00 00 00 00 00 00 00 00 00 00 00 fe 00 4c
    50 30 37 39 51 58 31 2d 53 50 30 56 00 00 00 fc
    00 43 6f 6c 6f 72 20 4c 43 44 0a 20 20 20 00 3f

    ----------------

    Block 0, Base EDID:
    ```

```
  EDID Structure Version & Revision: 1.0
  Vendor & Product Identification:
    Manufacturer: ETC
    Model: 0
    Made in: week 4 of 2013
  Basic Display Parameters & Features:
    Digital display
    Maximum image size: 16 cm x 12 cm
    Gamma: 2.20
    Monochrome or grayscale display
    Default (sRGB) color space is primary color space
    First detailed timing is the preferred timing
  Color Characteristics:
    Red  : 0.6396, 0.3300
    Green: 0.2998, 0.6005
    Blue : 0.1484, 0.0585
    White: 0.3134, 0.3291
  Established Timings I & II: none
  Standard Timings: none
  Detailed Timing Descriptors:
    DTD 1:  1536x2048   60.600861 Hz   3:4    125.323 kHz    202.020000
MHz (120 mm x 160 mm)
                 Hfront   12 Hsync  16 Hback   48 Hpol N
                 Vfront    8 Vsync   4 Vback    8 Vpol N
    Dummy Descriptor:
    Alphanumeric Data String: 'LP079QX1-SP0V'
    Display Product Name: 'Color LCD'
Checksum: 0x3f
```

- 若当前 SDK 中未支持 edid-decode 工具，则可以用以下指令保存为二进制文件后再解析查看：

```
cat /sys/class/drm/card0-eDP-1/edid > /data/edid.bin
```

Panel "lg,lp079qx1-sp0v" 的 EDID 通过网站 https://hverkuil.home.xs4all.nl/edid-decode/edid-decode.html 解析结果为：

```
edid-decode (hex):

00 ff ff ff ff ff ff 00 16 83 00 00 00 00 00 00
04 17 01 00 a5 10 0c 78 06 ef 05 a3 54 4c 99 26
0f 50 54 00 00 00 01 01 01 01 01 01 01 01 01 01
01 01 01 01 01 01 ea 4e 00 4c 60 00 14 80 0c 10
84 00 78 a0 00 00 00 18 00 00 00 10 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 fe 00 4c
50 30 37 39 51 58 31 2d 53 50 30 56 00 00 00 fc
00 43 6f 6c 6f 72 20 4c 43 44 0a 20 20 20 00 3f

----------------

Block 0, Base EDID:
  EDID Structure Version & Revision: 1.0
  Vendor & Product Identification:
    Manufacturer: ETC
    Model: 0
    Made in: week 4 of 2013
```

```
  Basic Display Parameters & Features:
    Digital display
    Maximum image size: 16 cm x 12 cm
    Gamma: 2.20
    Monochrome or grayscale display
    Default (sRGB) color space is primary color space
    First detailed timing is the preferred timing
  Color Characteristics:
    Red  : 0.6396, 0.3300
    Green: 0.2998, 0.6005
    Blue : 0.1484, 0.0585
    White: 0.3134, 0.3291
  Established Timings I & II: none
  Standard Timings: none
  Detailed Timing Descriptors:
    DTD 1:  1536x2048   60.600861 Hz   3:4    125.323 kHz    202.020000
MHz (120 mm x 160 mm)
                 Hfront   12 Hsync  16 Hback   48 Hpol N
                 Vfront    8 Vsync   4 Vback    8 Vpol N
    Dummy Descriptor:
    Alphanumeric Data String: 'LP079QX1-SP0V'
    Display Product Name: 'Color LCD'
Checksum: 0x3f

----------------

Preferred Video Timing if only Block 0 is parsed:
  DTD   1:  1536x2048   60.600861 Hz   3:4    125.323 kHz    202.020000
MHz (120 mm x 160 mm)
                 Hfront   12 Hsync  16 Hback   48 Hpol N
                 Vfront    8 Vsync   4 Vback    8 Vpol N

----------------

Native Video Resolution:
  1536x2048

----------------

edid-decode SHA: dc3a7b9f5453 2025-04-29 13:26:06

Warnings:

Block 0, Base EDID:
  EDID Structure Version & Revision: EDID 1.0 is deprecated, do not use.

Failures:

Block 0, Base EDID:
  Basic Display Parameters & Features: Digital Video Interface Standard
set to reserved value 0x25.
  Basic Display Parameters & Features: sRGB is signaled, but the
chromaticities do not match.
  Detailed Timing Descriptor #1: Mismatch of image size 120x160 mm vs
display size 160x120 mm.
  Display Descriptor #2: Has descriptor blocks other than detailed
timings.
```

```
  Display Descriptor #3: Has descriptor blocks other than detailed
timings.
  Display Descriptor #4: Has descriptor blocks other than detailed
timings.
EDID:
  Base EDID: The DTD max image size is 120x160mm, which is larger than
the display size 160.0x120.0mm.

EDID conformity: FAIL
```

- eabled：设备的使能状态。

```
console:/ # cat /sys/class/drm/card0-eDP-1/enabled
enabled
```

- modes：支持的分辨率列表。

```
console:/ # cat /sys/class/drm/card0-eDP-1/modes
1536x2048
```

- status：设备的连接状态。

```
cat /sys/class/drm/card0-eDP-1/status
connected
```

5.2 查看 Controller/PHY 寄存器

eDP 各平台 Controller/PHY 的基址如下：

	RK3288	RK3399	RK3568	RK3588	RK3576
Controller	0xff970000	0xff970000	0xfe0c0000	0xfdec0000/0xfded0000	0x27dc0000
PHY	0xff970600	0xff970600	0xfdcb0000	0xfed60000/0xfed70000	0x2b000000

Dump Controller 寄存器的指令（以 RK3576 为例）：

```
io -4 -l 0x400 0x27dc0000
```

Dump PHY 寄存器的指令需要按照平台区分：

- RK3288/RK3399

```
io -4 -l 0x200 0xff970600
```

- RK3568

```
io -4 -l 0x100 0xfdcb0000
```

- RK3588/RK3576

```
cat /sys/kernel/debug/regmap/2b000000.phy/registers
```

5.3 查看 VOP 状态

详见文档《Rockchip_Developer_Guide_DRM_Display_Driver_CN》的《dump 当前的显示状态》章节。

5.4 查看 Clock 配置

详见文档《Rockchip_Developer_Guide_DRM_Display_Driver_CN》的《查看当前显示时钟》章节。

5.5 强制开关 eDP

以 RK3576 平台为例，可以通过《查看 DRM Connector 状态》中介绍的 status 节点来强制开关 eDP 显示通路：

```
console:/ # echo off > /sys/class/drm/card0-eDP-1/status
console:/ # [  480.579852][  T333] rockchip-vop2 27d00000.vop:
[drm:vop2_crtc_atomic_disable] Crtc atomic disable vp1
[  480.589443][  T333] [WLAN_RFKILL]: wlan_early_suspend :enter
......
console:/ # echo on > /sys/class/drm/card0-eDP-1/status
console:/ # [  503.199908][  T333] [WLAN_RFKILL]: wlan_late_resume :enter
[  503.199952][  T333] rockchip-vop2 27d00000.vop: [drm:vop2_crtc_atomic_enable]
Update mode to 1536x2048p60, type: 14(if:eDP0, flag:0x0) for vp1 dclk: 200000000
[  503.200223][  T333] rockchip-vop2 27d00000.vop: [drm:vop2_crtc_atomic_enable]
set dclk_vp1 to 200000000, get 200000000
```

5.6 读写 DPCD

DPCD 读写通过 drm_dp_aux_dev 实现，驱动详见 /drivers/gpu/drm/drm_dp_aux_dev.c。

开启 CONFIG_DRM_DP_AUX_CHARDEV 配置项后，可以通过以下指令进行 DPCD 的读取和写入：

读取起始地址为 0x00000 的 16 个 DPCD 寄存器的值

```
console:/ # dd if=/dev/drm_dp_aux0 bs=1 skip=$((0x00000)) count=16 status=none |
od -tx1
0000000 11 06 84 41 00 00 01 c0 02 00 00 00 0f 03 00 00
0000020
```

- if：指定 aux 节点。当注册两个 eDP 接口时，会有 /dev/drm_dp_aux0 和 /dev/drm_dp_aux1 两个节点供使用。
- skip：配置 DPCD 寄存器起始地址。
- count：配置 DPCD 寄存器读取数量。

将 0x0a 和 0x80 两个值写入 0x100 起始的两个 DPCD 寄存器

```
console:/ # echo -e -n "\x0a\x80" | dd of=/dev/drm_dp_aux0 bs=1 seek=$((0x100))
count=2 status=none
```

- of：指定 aux 节点。当注册两个 eDP 接口时，会有 /dev/drm_dp_aux0 和 /dev/drm_dp_aux1 两个节点供使用。
- seek：配置 DPCD 寄存器起始地址。
- count：配置 DPCD 寄存器写入数量。

5.7 调整 DRM Log 类别

DRM 对于 Debug Log 等级的解释详见 `drivers/gpu/drm/drm_print.c`

```
/*
 * __drm_debug: Enable debug output.
 * Bitmask of DRM_UT_x. See include/drm/drm_print.h for details.
 */
unsigned long __drm_debug;
EXPORT_SYMBOL(__drm_debug);

MODULE_PARM_DESC(debug, "Enable debug output, where each bit enables a debug
category.\n"
"\t\tBit 0 (0x01)  will enable CORE messages (drm core code)\n"
"\t\tBit 1 (0x02)  will enable DRIVER messages (drm controller code)\n"
"\t\tBit 2 (0x04)  will enable KMS messages (modesetting code)\n"
"\t\tBit 3 (0x08)  will enable PRIME messages (prime code)\n"
"\t\tBit 4 (0x10)  will enable ATOMIC messages (atomic code)\n"
"\t\tBit 5 (0x20)  will enable VBL messages (vblank code)\n"
"\t\tBit 7 (0x80)  will enable LEASE messages (leasing code)\n"
"\t\tBit 8 (0x100) will enable DP messages (displayport code)");
```

可以根据需求调整 DRM Log 的类别，若想要查看 DPCD 读写相关的打印，则输入如下指令：

```
echo 0x100 > /sys/module/drm/parameters/debug
```

也可以直接修改上文 __drm_debug 变量值（注意：此方法不适用于 GKI 版本 SDK）：

```
diff --git a/drivers/gpu/drm/drm_print.c b/drivers/gpu/drm/drm_print.c
index 5b93c11895bb..8da96cde8308 100644
--- a/drivers/gpu/drm/drm_print.c
+++ b/drivers/gpu/drm/drm_print.c
@@ -39,7 +39,7 @@
  * __drm_debug: Enable debug output.
  * Bitmask of DRM_UT_x. See include/drm/drm_print.h for details.
  */
-unsigned long __drm_debug;
+unsigned long __drm_debug = 0x100;
 EXPORT_SYMBOL(__drm_debug);
```

6. FAQ

6.1 带宽计算方法

计算指定 display mode 下 eDP 每条 Lane 需要支持的带宽，公式如下：

$$bandwidth_per_lane = pixel_clk * bit_per_pixel * 1.25 / lane_count$$

其中，`bit_per_pixel`是每个 pixel 的 bit 数，`1.25`是 eDP 协议规定的 Lane 数据编码转换效率，`lane_count`是可用的 Lane 的数量，最终的计算结果`bandwith_per_lane`即每条 Lane 需要提供的最小带宽，如果当前的 Lane rate 比需要的最小带宽小，相应 pixel clk 的 display mode 就会被 eDP 驱动程序过滤掉。

例如，对于一个 Lane 数量为 2，分辨率是 2K@60Hz 且 pixel clk 为 297MHz 的屏幕，采用 RGB888 格式传输需要每条 Lane 支持的最小带宽为:

$$bandwidth_per_lane = 297 * 24 * 1.25 / 2 = 4.46 Gbps/lane$$

参考《功能特性》中的介绍，可以判断 RK3399/RK3588/RK3576 平台可以支持该屏幕，而 RK3288/RK3568 不支持。

6.2 DCLK 精度问题

在 U-Boot 阶段通过以下 Log 来确认预期和实际 DCLK 是否有差异（以 RK3588S EVB1 为例）：

```
edp@fdec0000:  detailed mode clock 200000 kHz, flags[a] //expected dclk rate
    H: 1536 1548 1564 1612
    V: 2048 2056 2060 2068
bus_format: 100e
VOP update mode to: 1536x2048p60, type: eDP0 for VP2
VP2 set crtc_clock to 200000KHz                         //actual dclk rate
```

在 Kernel 阶段通过指令 cat /sys/kernel/debug/dri/0/summary 进行确认：

```
console:/ # cat /sys/kernel/debug/dri/0/summary
Video Port1: DISABLED
Video Port2: ACTIVE
    Connector:eDP-1     Encoder: TMDS-463
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[f] SDR[0] color-encoding[BT.709] color-
range[Full]
    Display mode: 1536x2048p60
        dclk[200000 kHz] real_dclk[200000 kHz] aclk[500000 kHz] type[48] flag[a]
//dclk[expected] real_dclk[actual]
        H: 1536 1548 1564 1612
        V: 2048 2056 2060 2068
        Fixed H: 1536 1548 1564 1612
        Fixed V: 2048 2056 2060 2068
    Cluster2-win0: ACTIVE
        win_id: 4
        format: AB24 little-endian (0x34324241)_AFBC-16x16 pixel_blend_mode[0]
glb_alpha[0xff]
        color: SDR[0] color-encoding[BT.709] color-range[Full]
        rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
        csc: y2r[0] r2y[0] csc mode[0]
        zpos: 0
```

```
        src: pos[0, 0] rect[1536 x 2048]
        dst: pos[0, 0] rect[1536 x 2048]
        buf[0]: addr: 0x00000000f16bb000 pitch: 6144 offset: 0
```

如遇到实际 DCLK 与预期不符，先根据 Panel datasheet 确认实际分配的 DCLK 是否满足要求，在满足 datasheet 要求、显示正常且帧率符合预期的情况下，可以忽略 DCLK 与预期不符问题：

- eDP Panel 通常对 Pixel Clock 具备一定的“容忍度”，在一定误差范围内是可以正常显示且保证帧率的。
- 每个芯片平台的 PLL 是有限的，无法保证每个模块的 DCLK 都能精确分配，原则上保证“就近分频”，详见文档《Rockchip_Developer_Guide_Clock_CN》。

如已与屏厂确认 Panel 对 Pixel Clock 精度十分敏感，且由于实际 DCLK 与预期不符而导致了显示异常问题，可以参考文档《Rockchip_Developer_Guide_DRM_Display_Driver_CN》的《VOP 平台 DCLK 时钟》章节将相应 VOP/VP 的 DCLK 配置为显示独占的 PLL 来保证精度。

6.3 AUX 报错分析及解决方法

```
/* DEMO1 */
[   11.591242] rockchip-dp fe0c0000.edp: AUX CH error happened: 0x2 (1)
[   11.592663] rockchip-dp fe0c0000.edp: AUX CH error happened: 0x2 (1)
[   11.594057] rockchip-dp fe0c0000.edp: AUX CH error happened: 0x2 (1)
/* DEMO2 */
console:/ # dmesg | grep edp
[ 3.236549] rockchip-dp fdec0000.edp: failed to read max link rate
[ 3.260422] rockchip-dp fdec0000.edp: failed to read max link rate
[ 3.284163] rockchip-dp fdec0000.edp: failed to read max link rate
```

可以在以下方面进行检查和确认：

6.3.1 检查 PHY 配置

详见《PHY DTS 配置》，需确保 eDP0/eDP1 对应的 PHY 节点被正确使能。

6.3.2 检查 Panel 配置

详见《Panel DTS 配置》，需要确保 Panel DTS 的各个配置符合 Datasheet 的要求。

还应检查 Power 配置是否符合 Panel 要求（具体要求参考 Panel Datasheet），详见文档《Rockchip_Developer_Guide_Linux_IO_DOMAIN_CN》。

需要注意的是，Panel Power-supply 建议由主控控制，这样对于特殊的异常情况还有复位的补救方法。

6.3.3 确认 HPD 状态及上电时序

确认 HPD 的配置方式（详见《HPD 配置》），Force HPD、Function Pin HPD 和 GPIO HDP 需三者选其一。

- Force HPD

Force HPD 模式下由于没有 HPD 信号将 Sink 端的就绪状态通知到 Source 端，所以可能会存在上电时序不符合 Datasheet 要求的情况。

时序配置应参考《Panel DTS 配置》章节通过示波器确认 enable-delay-ms/reset-delay-ms/prepare-delay-ms/unprepare-delay-ms/disable-delay-ms 配置。

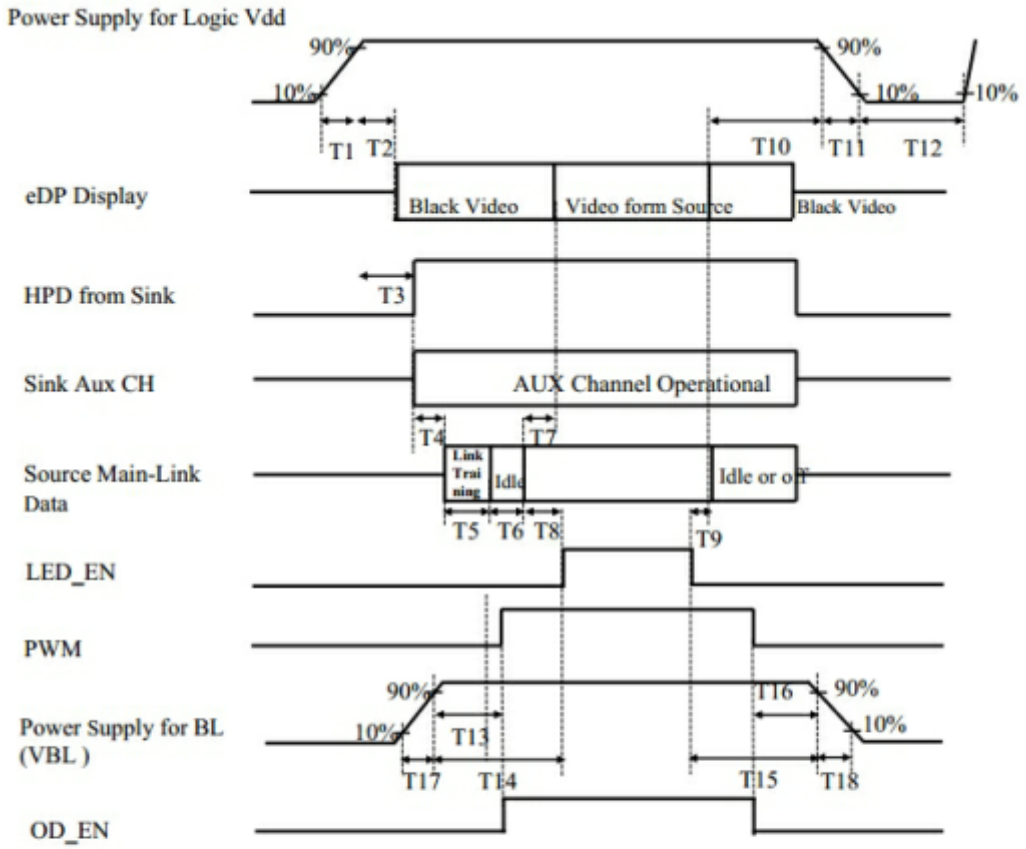


Figure 18. Power Sequence

- 0.5ms ≤ T1 ≤ 10 ms
- 0ms < T2 ≤ 200 ms
- 0ms < T3 ≤ 200 ms
- T4+T5+T6+T8>80ms
- 0ms < T7 ≤ 50ms
- 50ms < T8
- 0ms < T9
- 100ms < T10 < 500 ms
- 0.5ms ≤ T11 ≤ 10 ms
- 500ms ≤ T12
- 0ms < T13
- 0ms < T14
- 0ms < T15
- 0ms < T16
- 0.5ms ≤ T17
- 0.5ms ≤ T18

对于各时序均已符合 Datasheet 要求仍有 AUX 报错的情况，可以适当延长 T3（如上图，即便超过要求的≤200）即 prepare-delay-ms 来确保 Sink 端在 Source 发起 AUX 通信前处于就绪状态，如：

```
&panel {
    prepare-delay-ms = <500>;
}
```

- GPIO HPD

GPIO HPD 模式下除上述上电时序检查外，还需确认指定的 HPD PIN 是否有被其他模块占用，可使用指令 `cat /d/gpio` 确认：

```
console:/ # cat /d/gpio
gpiochip1: GPIOs 32-63, parent: platform/ff730000.gpio1, gpio1:
 gpio-33  (                    |?                   ) out lo
 gpio-34  (                    |int-n              ) in  hi
 gpio-45  (                    |enable             ) out lo
 gpio-46  (                    |vsel               ) out lo
 gpio-49  (                    |vsel               ) out lo
 gpio-50  (                    |vbus-5v            ) out lo
 gpio-55  (                    |reset              ) out lo
... ...
gpiochip5: GPIOs 511-511, parent: platform/rk805-pinctrl, rk817-gpio, can sleep:
```

另外，GPIO HPD 还可以通过 Pin 下拉配置来确保 HPD 信号的稳定识别，详见《HPD 配置》。

6.3.4 检查 AUX 硬件设计

- 确认 AUX 差分线正负极是否有接反，若因硬件设计疏漏而反向，参考下述修改：

  U-Boot：

```
diff --git a/drivers/video/drm/analogix_dp_reg.c
b/drivers/video/drm/analogix_dp_reg.c
index 292a5076705..373189e9157 100644
--- a/drivers/video/drm/analogix_dp_reg.c
+++ b/drivers/video/drm/analogix_dp_reg.c
@@ -1241,7 +1241,7 @@ ssize_t analogix_dp_aux_transfer(struct drm_dp_aux
*aux, struct drm_dp_aux_msg *
        }

        /* Enable AUX CH operation */
-       reg = AUX_EN;
+       reg = AUX_EN | 0x4;

        /* Zero-sized messages specify address-only transactions. */
        if (msg->size < 1)
```

  Kernel：

```
diff --git a/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
b/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
index 810b5a2f6041..2dbde10033c5 100644
--- a/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
+++ b/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
@@ -1202,7 +1202,7 @@ ssize_t analogix_dp_transfer(struct analogix_dp_device
*dp,
        }

        /* Enable AUX CH operation */
-       reg = AUX_EN;
+       reg = AUX_EN | 0x4;

        /* Zero-sized messages specify address-only transactions. */
        if (msg->size < 1)
```

- 对于 eDP v1.2 及以上版本的 AUX 链路硬件设计，应去掉 AUX 差分线的上下拉电阻，若仍有 AUX 报错异常，可以尝试降低 AUX 信号线串联电容。

6.4 Link Training 报错分析及解决方法

eDP Link Training 报错如下：

```
# Link Training CR(Clock Recovery) 阶段失败
[ 1410.198900][ T318] rockchip-dp fdec0000.edp: CR Max reached (0,3,0)
[ 1410.202213][ T318] rockchip-dp fdec0000.edp: LT CR failed!
[ 1410.202227][ T318] rockchip-dp fdec0000.edp: eDP link training failed (-5)
[ 1410.202238][ T318] rockchip-dp fdec0000.edp: unable to do link train, ret=-5

# Link Training EQ(Channel Equalization) 阶段失败
[ 6673.100385][ T318] rockchip-dp fdec0000.edp: EQ Max loop
[ 6673.101511][ T318] rockchip-dp fdec0000.edp: LT EQ failed!
[ 6673.101515][ T318] rockchip-dp fdec0000.edp: eDP link training failed (-5)
[ 6673.101517][ T318] rockchip-dp fdec0000.edp: unable to do link train, ret=-5
[ 6673.101522][ T318] [drm:analogix_dp_bridge_atomic_enable] ERROR dp commit
error, ret = -5
[ 6673.101533][ T318] rockchip-dp fdec0000.edp: failed to set bridge, retry: 0
```

eDP Link Training 的主循环代码大致如下，不同版本会有差异：

```
while (!retval && !training_finished) {
    switch (dp->link_train.lt_state) {
        case START:
            retval = analogix_dp_link_start(dp);
            if (retval)
                dev_err(dp->dev, "LT link start failed!\n");
            break;
        case CLOCK_RECOVERY:
            retval = analogix_dp_process_clock_recovery(dp);
            if (retval)
                dev_err(dp->dev, "LT CR failed!\n");
            break;
        case EQUALIZER_TRAINING:
            retval = analogix_dp_process_equalizer_training(dp);
            if (retval)
                dev_err(dp->dev, "LT EQ failed!\n");
            break;
        case FINISHED:
            training_finished = 1;
            break;
        case FAILED:
            return -EREMOTEIO;
    }
}
```

eDP Link Training 驱动上可分为 START、CLOCK_RECOVERY 和 EQUALIZER_TRAINING 三个阶段：

- START：Training 起始参数初始化。
- CLOCK_RECOVERY：链路时钟恢复。
- EQUALIZER_TRAINING：链路均衡。

可以从 Log 中判断 Link Training 在哪个阶段出错。

6.4.1 确认 Panel 协议支持

对比《功能特性》章节中说明的 RK 各平台 eDP 接口的协议支持情况与 eDP Panel Datasheet 标明的协议版本，eDP 接口支持的协议版本必须大于等于 Panel 的协议版本。

6.4.2 检查 Main Link 硬件设计

- 确认 Main Link 各条 Lane 在 Source 和 Sink 两端是否有一一对应，若硬件设计有误，可参考以下修改调整 Lane 的对应关系：

```
&edp0 {
    data-lanes = <3 2 1 0>;
};
```

- 确认 Main Link 硬件设计是否符合规范，参考 RK 平台官方的硬件设计指南，以 RK3576 为例为《Rockchip RK3576 Hardware Design Guide》。

6.4.3 检查硬件上的屏线和接口是否合规

屏线和接口的规范及标准需与屏厂确认，常见要求为：

- 屏线长度不超过 30cm
- 屏线硬件上应有屏蔽设计
- 屏线和接口的连接需正确且稳定

6.4.4 确认 HPD 状态及上电时序

同上文《确认 HPD 状态及上电时序》。

6.4.5 修改预加重和电压摆幅配置

PE（Pre-emphasic，预加重）和 VS（Voltage Swing，电压摆幅）配置是影响 eDP 信号质量的重要因素，两者的配置组合如下：

Voltage Swing Level	**Pre-emphasis Level**			
	Level 0 Vdiff_pre_pp	**Level 1 Vdiff_pre_pp**	**Level 2 Vdiff_pre_pp**	**Level 3 Vdiff_pre_pp**
0	Required	Required	Required	Optional
1	Required	Required	Required	Not allowed
2	Required	Required	Not allowed	Not allowed
3	Optional	Not allowed	Not allowed	Not allowed

6.4.5.1 修改起始配置

eDP 协议规定的起始配置为 PE Level 0/ VS Level 0，可以通过下述 PE/VS 起始配置的修改来实现不同组合的尝试，参数的宏定义如下：

```
# define DP_TRAIN_MAX_SWING_REACHED     (1 << 2)
# define DP_TRAIN_VOLTAGE_SWING_LEVEL_0 (0 << 0)
# define DP_TRAIN_VOLTAGE_SWING_LEVEL_1 (1 << 0)
# define DP_TRAIN_VOLTAGE_SWING_LEVEL_2 (2 << 0)
# define DP_TRAIN_VOLTAGE_SWING_LEVEL_3 (3 << 0)

# define DP_TRAIN_PRE_EMPHASIS_MASK     (3 << 3)
# define DP_TRAIN_PRE_EMPH_LEVEL_0      (0 << 3)
# define DP_TRAIN_PRE_EMPH_LEVEL_1      (1 << 3)
# define DP_TRAIN_PRE_EMPH_LEVEL_2      (2 << 3)
# define DP_TRAIN_PRE_EMPH_LEVEL_3      (3 << 3)
```

Uboot 参考修改如下：

```
diff --git a/drivers/video/drm/analogix_dp.c b/drivers/video/drm/analogix_dp.c
index c4b50018baa..0afb116104f 100644
--- a/drivers/video/drm/analogix_dp.c
+++ b/drivers/video/drm/analogix_dp.c
@@ -240,8 +240,8 @@ static int analogix_dp_link_start(struct analogix_dp_device
*dp)
        /* Set TX voltage-swing and pre-emphasis to minimum */
        for (lane = 0; lane < lane_count; lane++)
                dp->link_train.training_lane[lane] =
-                               DP_TRAIN_VOLTAGE_SWING_LEVEL_0 |
-                               DP_TRAIN_PRE_EMPH_LEVEL_0;
+                               DP_TRAIN_VOLTAGE_SWING_LEVEL_2 |
+                               DP_TRAIN_PRE_EMPH_LEVEL_1;
        analogix_dp_set_lane_link_training(dp);

        /* Set training pattern 1 */
@@ -254,8 +254,8 @@ static int analogix_dp_link_start(struct analogix_dp_device
*dp)
                return retval;

        for (lane = 0; lane < lane_count; lane++)
-               buf[lane] = DP_TRAIN_PRE_EMPH_LEVEL_0 |
-                           DP_TRAIN_VOLTAGE_SWING_LEVEL_0;
+               buf[lane] = DP_TRAIN_PRE_EMPH_LEVEL_1 |
+                           DP_TRAIN_VOLTAGE_SWING_LEVEL_2;

        retval = drm_dp_dpcd_write(&dp->aux, DP_TRAINING_LANE0_SET, buf,
                                   lane_count);
```

Kernel 参考修改如下：

```
diff --git a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
index 4015073bbb39..10407e67c95f 100644
--- a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
+++ b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
```

```
@@ -435,8 +435,8 @@ static int analogix_dp_link_start(struct analogix_dp_device
*dp)
        /* Set TX voltage-swing and pre-emphasis to minimum */
        for (lane = 0; lane < lane_count; lane++)
                dp->link_train.training_lane[lane] =
-                                       DP_TRAIN_VOLTAGE_SWING_LEVEL_0 |
-                                       DP_TRAIN_PRE_EMPH_LEVEL_0;
+                                       DP_TRAIN_VOLTAGE_SWING_LEVEL_2 |
+                                       DP_TRAIN_PRE_EMPH_LEVEL_1;
        analogix_dp_set_lane_link_training(dp);

        /* Set training pattern 1 */
@@ -450,8 +450,8 @@ static int analogix_dp_link_start(struct analogix_dp_device
*dp)
                return retval;

        for (lane = 0; lane < lane_count; lane++)
-               buf[lane] = DP_TRAIN_PRE_EMPH_LEVEL_0 |
-                           DP_TRAIN_VOLTAGE_SWING_LEVEL_0;
+               buf[lane] = DP_TRAIN_PRE_EMPH_LEVEL_2 |
+                           DP_TRAIN_VOLTAGE_SWING_LEVEL_1;

        retval = drm_dp_dpcd_write(&dp->aux, DP_TRAINING_LANE0_SET, buf,
                                   lane_count);
```

6.4.5.2 添加设置打印

需要注意的是，设置的 Voltage_swing 和 Pre-emphasis 起始参数进行首轮 Link Training 后若失败，会根据 Panel 建议的参数进行下一轮 Link Training。

可以参考以下添加打印确定后续设置的预加重和电压摆幅参数，Uboot 参考如下：

```
diff --git a/drivers/video/drm/analogix_dp_reg.c
b/drivers/video/drm/analogix_dp_reg.c
index d2cb10cc0e..c96495f5e9 100644
--- a/drivers/video/drm/analogix_dp_reg.c
+++ b/drivers/video/drm/analogix_dp_reg.c
@@ -1002,6 +1002,7 @@ void analogix_dp_set_lane_link_training(struct
analogix_dp_device *dp)
                     DP_TRAIN_PRE_EMPHASIS_SHIFT;
                phy_cfg.dp.voltage[lane] = vs;
                phy_cfg.dp.pre[lane] = pe;
+               printf("[Link Training] vs = %d pe = %d\n", vs, pe);
        }

        phy_cfg.dp.lanes = dp->link_train.lane_count;
```

Kernel 参考如下：

```
diff --git a/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
b/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
index 42d1cbe28b40..ea5657770cfb 100644
--- a/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
+++ b/drivers/gpu/drm/bridge/analogix/analogix_dp_reg.c
```

```
@@ -733,10 +733,14 @@ void analogix_dp_set_lane_link_training(struct
analogix_dp_device *dp)
        u8 lane;
        int ret;

-       for (lane = 0; lane < dp->link_train.lane_count; lane++)
+       for (lane = 0; lane < dp->link_train.lane_count; lane++) {
                analogix_dp_write(dp,
                                  ANALOGIX_DP_LN0_LINK_TRAINING_CTL + 4 * lane,
                                  dp->link_train.training_lane[lane]);
+               printk("[Link Training] vs = %d pe = %d\n",
+                                 (dp->link_train.training_lane[lane] &
DP_TRAIN_VOLTAGE_SWING_MASK) >> DP_TRAIN_VOLTAGE_SWING_SHIFT,
+                                 (dp->link_train.training_lane[lane] &
DP_TRAIN_PRE_EMPHASIS_MASK) >> DP_TRAIN_PRE_EMPHASIS_SHIFT);
+       }

        if (dp->phy) {
                union phy_configure_opts phy_cfg;
```

6.4.6 降速尝试

带宽计算方法详见上文《带宽计算方法》。

在带宽满足的情况下，可以进行降低 Link Rate 的尝试，参考 DTS 配置如下：

```
&edp {
    force-hpd;
    status = "okay";
    ports {
        port@1 {
            reg = <1>;
            edp_out: endpoint {
                link-frequencies = <2700000000>;
                remote-endpoint = <&panel_in>;
            };
        };
    };
};
```

或者通过在驱动中强行指定 Link Rate 的方式，Link Rate 宏定义如下：

```
# define DP_LINK_BW_1_62            0x06
# define DP_LINK_BW_2_7             0x0a
# define DP_LINK_BW_5_4             0x14
# define DP_LINK_BW_8_1             0x1e
```

U-Boot 参考修改：

```
diff --git a/drivers/video/drm/analogix_dp.c b/drivers/video/drm/analogix_dp.c
index c4b50018baa..35daf949017 100644
--- a/drivers/video/drm/analogix_dp.c
+++ b/drivers/video/drm/analogix_dp.c
@@ -665,6 +665,7 @@ static int analogix_dp_get_max_rx_bandwidth(struct
analogix_dp_device *dp,

                *bandwidth = data;
        }
+       *bandwidth = DP_LINK_BW_1_62;

        return 0;
 }
```

Kernel 参考修改：

```
diff --git a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
index 4015073bbb39..f70898e0319e 100644
--- a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
+++ b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
@@ -851,6 +851,7 @@ static int analogix_dp_get_max_rx_bandwidth(struct
analogix_dp_device *dp,

                *bandwidth = data;
        }
+       *bandwidth = DP_LINK_BW_1_62;

        return 0;
 }
```

6.4.7 增加 CR/EQ 阶段的 Retry 次数

eDP 协议中 CR/EQ 阶段 Retry 次数最大为 5，而一些特定的屏幕可能需要更多次的 Retry 来保证 Link Training 的成功。

U-Boot 参考修改如下：

```
diff --git a/drivers/video/drm/analogix_dp.h b/drivers/video/drm/analogix_dp.h
index c6cb113190e..f9312f7391d 100644
--- a/drivers/video/drm/analogix_dp.h
+++ b/drivers/video/drm/analogix_dp.h
@@ -460,9 +460,9 @@
 /* ANALOGIX_DP_LINK_POLICY */
 #define ALTERNATE_SR_ENABLE                    (0x1 << 7)

-#define DP_TIMEOUT_LOOP_COUNT 100
-#define MAX_CR_LOOP 5
-#define MAX_EQ_LOOP 5
+#define DP_TIMEOUT_LOOP_COUNT 200
+#define MAX_CR_LOOP 10
+#define MAX_EQ_LOOP 10

 /* I2C EDID Chip ID, Slave Address */
```

```
 #define I2C_EDID_DEVICE_ADDR                    0x50
```

Kernel 参考修改如下：

```
diff --git a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.h
b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.h
index 879a56eab2b7..8d493a0f76d1 100644
--- a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.h
+++ b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.h
@@ -13,9 +13,9 @@
 #include <drm/drm_crtc.h>
 #include <drm/drm_bridge.h>

-#define DP_TIMEOUT_LOOP_COUNT 100
-#define MAX_CR_LOOP 5
-#define MAX_EQ_LOOP 5
+#define DP_TIMEOUT_LOOP_COUNT 200
+#define MAX_CR_LOOP 10
+#define MAX_EQ_LOOP 10
 #define MAX_PLL_LOCK_LOOP 5

 /* Training takes 22ms if AUX channel comm fails. Use this as retry interval */
```

6.4.8 增加复位 Panel 操作

U-Boot 参考修改如下:

```
diff --git a/drivers/video/drm/analogix_dp.c b/drivers/video/drm/analogix_dp.c
index c4b50018baa..91ff3fa2ae6 100644
--- a/drivers/video/drm/analogix_dp.c
+++ b/drivers/video/drm/analogix_dp.c
@@ -715,6 +715,12 @@ static int analogix_dp_init_training(struct
analogix_dp_device *dp,
        /* All DP analog module power up */
        analogix_dp_set_analog_power_down(dp, POWER_ALL, 0);

+       if (dp->connector.panel->funcs->unprepare)
+               dp->connector.panel->funcs->unprepare(dp->connector.panel);
+
+       if (dp->connector.panel->funcs->prepare)
+               dp->connector.panel->funcs->prepare(dp->connector.panel);
+
        return 0;
 }
```

Kernel 参考修改如下：

```
diff --git a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
index 4015073bbb39..4dfc04a6ccae 100644
--- a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
+++ b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
@@ -910,6 +910,11 @@ static int analogix_dp_full_link_train(struct
analogix_dp_device *dp,
```

```
        /* All DP analog module power up */
        analogix_dp_set_analog_power_down(dp, POWER_ALL, 0);

+       if (dp->plat_data->panel) {
+               analogix_dp_panel_unprepare(dp);
+               analogix_dp_panel_prepare(dp);
+       }
+
        dp->link_train.lt_state = START;

        /* Process here */
```

6.4.9 尝试强制输出

若上述方法均无法使 Link Training 成功，还可以尝试强制输出以确认问题。

U-Boot 参考修改如下：

```
diff --git a/drivers/video/drm/analogix_dp.c b/drivers/video/drm/analogix_dp.c
index c4b50018baa..5cf45165698 100644
--- a/drivers/video/drm/analogix_dp.c
+++ b/drivers/video/drm/analogix_dp.c
@@ -1035,6 +1035,7 @@ static int analogix_dp_connector_enable(struct
rockchip_connector *conn,
        u32 fmt_id;
        u32 val;
        int ret;
+       int i;

        drm_mode_copy(&video->mode, &conn_state->mode);

@@ -1097,8 +1098,11 @@ static int analogix_dp_connector_enable(struct
rockchip_connector *conn,
        ret = analogix_dp_set_link_train(dp, dp->video_info.max_lane_count,
                                         dp->video_info.max_link_rate);
        if (ret) {
-               dev_err(dp->dev, "unable to do link train\n");
-               return ret;
+               dev_err(dp->dev, "unable to do link train, using default
strength\n");
+               for (i = 0; i < dp->link_train.lane_count; i++) {
+                       dp->link_train.training_lane[i] =
DP_TRAIN_VOLTAGE_SWING_LEVEL_0 | DP_TRAIN_PRE_EMPH_LEVEL_0;
+               }
+               analogix_dp_set_lane_link_training(dp);
        }

        ret = analogix_dp_enable_scramble(dp, 1);
```

Kernel 参考修改如下：

```
diff --git a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
index 4015073bbb39..b97cd40ef205 100644
--- a/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
```

```
+++ b/drivers/gpu/drm/bridge/analogix/analogix_dp_core.c
@@ -1341,6 +1341,7 @@ static int analogix_dp_commit(struct analogix_dp_device
*dp)
 {
        struct video_info *video = &dp->video_info;
        int ret;
+       int i;

        ret = drm_dp_read_dpcd_caps(&dp->aux, dp->dpcd);
        if (ret < 0) {
@@ -1362,8 +1363,10 @@ static int analogix_dp_commit(struct analogix_dp_device
*dp)

        ret = analogix_dp_train_link(dp);
        if (ret) {
-               dev_err(dp->dev, "unable to do link train, ret=%d\n", ret);
-               return ret;
+               dev_err(dp->dev, "unable to do link train, using default
strength\n");
+               for (i = 0; i < dp->link_train.lane_count; i++)
+                       dp->link_train.training_lane[i] =
DP_TRAIN_VOLTAGE_SWING_LEVEL_0 | DP_TRAIN_PRE_EMPH_LEVEL_0;
+               analogix_dp_set_lane_link_training(dp);
        }

        ret = analogix_dp_enable_scramble(dp, 1);
```

注意：上述强制输出方式与 eDP 协议规范不符，若可以点亮屏幕，仍可能存在隐患，建议与屏厂联系进一步分析。。

6.5 显示异常（Training 成功）分析及解决方法

若 Link Training 已经成功，仍为黑屏或显示异常，请依次尝试以下 DEBUG 方法。

6.5.1 开启自测模式

详见上文《Panel 自测模式》章节和《Controller 自测模式》章节。

6.5.2 确认 VOP 状态是否符合预期

详见上文《查看 VOP 状态》章节。

6.5.3 确认 Clock 配置是否符合预期

详见上文《查看 Clock 配置》章节。

6.6 如何优化信号质量

6.6.1 优化 PE/VS Level 配置

详见上文《修改 PE/VS 配置》章节。

6.6.2 开启 SSC 功能

SSC（Spread Spectrum Clocking，扩频时钟）可以降低电磁干扰（EMI）、降低电源噪声影响等，以起到优化信号完整性的作用。

RK 平台在 **Sink 端支持 SSC 功能的情况下**会默认开启该功能，SSC 功能的检测函数如下，可以添加打印看是否支持：

```
bool analogix_dp_ssc_supported(struct analogix_dp_device *dp)
{
    /* Check if SSC is supported by both sides */
    return dp->plat_data->ssc && dp->link_train.ssc;
}
```

如需要强制打开 SSC 功能，让上述函数返回 True 即可，Uboot 和 Kernel 都要同时修改。